
Software Requirements Specification

for

MoldMe

Redigido por:

8170538 – Miguel Ângelo Barbosa Ribeiro

8200489 - João Pedro Ferreira Teixeira

8210127 – Leonel da Silva de Carvalho

8210136 – Tiago André Cardoso Ribeiro

ESTG

2024-09-25

Tabela de Conteúdos

1. Introdução.....	1
1.1 Proposta de Sistema	1
1.2 Convenções do Documento	3
1.3 Sugestões de Leitura.....	4
1.4 Âmbito do Produto.....	5
1.5 Referências	6
2. Arquiteturas	7
2.1 Alto Nível (Diagrama de Pacotes)	7
2.2 Diagrama de Classes	9
3. Requisitos de Software.....	10
3.1 Requisitos Funcionais	10
3.2 Requisitos Não Funcionais	22
4. Diagramas UML.....	23
4.1 Casos de Uso	23
5. Product Backlog	27
5.1 Sprint#1 Backlog	29
5.2 Sprint#2 Backlog	30
5.3 Sprint#3 Backlog	31
6. Documentação da instância do GitLab.....	33
6.1 User Stories	33
6.2 Software Configuration Management Plan	33
6.3 Atividades CM	36
6.4 CM no Ciclo de Vida do Desenvolvimento	40
7. Evidências GitHub	41
8. Base de Dados.....	45
8.1 Desenho Conceptual	45
8.2 Modelo Lógico.....	55
8.3 Desenho Físico	64
9. Mockups.....	65
10. Testes	82
11. Evidências de requests.....	103
11.1 Postman.....	103
11.2 Swagger	104

Índice de Tabelas

Tabela 1 - RF 101.....	11
Tabela 2 - RF 102	11
Tabela 3 - RF 103.....	12
Tabela 4 - RF 104	13
Tabela 5 - RF 105.....	14
Tabela 7 - Entidades identificadas	47
Tabela 8 - Relacionamentos entre entidades.....	48
Tabela 9 - Atributos das Entidades.....	49
Tabela 10 - Documentação de Atributos.....	51

Tabela 11 – Atribuição de chaves primárias	52
--	----

Índice de Ilustrações

Figura 1 – Diagrama de Casos de Uso – Registo e autenticação.....	24
Figura 2– Diagrama de Casos de Uso – Gestão de Entidades	25
Figura 3 – Diagrama de Casos de Uso – Gestão de Projetos.....	26
Figura 4 – Identificação das labels.....	38
Figura 5 – Diagrama ER com entidades e tipos de relacionamentos.....	48
Figura 6– Diagrama ER com chaves primárias.....	52
Figura 7 – Diagrama Conceptual.....	54
Figura 8 – Desenho lógico.....	63
Figura 9 – HomePage	66
Figura 10 – Login – Web.....	67
Figura 11 – Recuperar Password – Web	68
Figura 12 – Recuperar Password – Mobile	68
Figura 13 – Tudo pronto (Login).....	69
Figura 14–Registo	69
Figura 15 – Escolher plano	70
Figura 16 – Pagamento de Plano	70
Figura 17 – Registo.....	71
Figura 18 – Perfil público da empresa.....	72
Figura 19 – Perfil Público de Funcionário com avaliações	73
Figura 20 – Editar perfil de funcionário – Mobile.....	74
Figura 21 – Editar perfil da Empresa.....	74
Figura 22 – Editar perfil de funcionário	74
Figura 23– Lista de funcionários de uma Empresa	75
Figura 24 – Lista de Projetos de uma Empresa.....	76
Figura 25 – Lista de Projetos de um funcionário	76
Figura 26 – Lista de projetos – Mobile.....	77
Figura 27 – Novo Projeto	77
Figura 28 – Proposta para Projeto.....	78
Figura 29– Página do Projeto	79
Figura 30 – Página do Projeto.....	80
Figura 31 – Upload de Documentos em Projeto	80
Figura 32– Chat de um Projeto – Mobile	81

Histórico de Revisão

Nome	Data	Razão para Alterações	Versão
João Pedro Teixeira & Miguel Ângelo Barbosa Ribeiro & Tiago Ribeiro & Leonel Carvalho	25/09/2024	Início do documento de especificação de software Redação de parte da secção de Introdução, excluindo os conteúdos que ainda não foram realizados, encontrando-se esta, portanto, inacabada, mas suficiente para a Milestone#1.	1.0
João Pedro Teixeira & Tiago Ribeiro & Leonel Carvalho & Miguel Ribeiro	19/10/2024	Documentação da Base de Dados Requisitos de sistema e Diagramas de casos de uso. Documentação da Instância do GitLab	2.0
João Pedro Teixeira & Tiago Ribeiro & Leonel Carvalho & Miguel Ribeiro	08/11/2024	Correções necessárias ao relatório. Atualização dos Diagramas Documentação dos TestCaseOutlines Evidências do uso do GitHub.	3.0
João Pedro Teixeira & Tiago Ribeiro & Leonel Carvalho & Miguel Ribeiro	06/12/2024	Correções necessárias ao relatório. Atualização do Diagrama de classes. Product Backlog	4.0

1. Introdução

1.1 Proposta de Sistema

O propósito deste Documento de Especificação de Requisitos (SRS) será definir requisitos funcionais e requisitos não funcionais para o MouldMe, versão 1.0.

Este produto é desenvolvido para alavancar a comunicação, gestão de projetos e colaboração entre empresas da indústria dos moldes, oferecendo uma plataforma onde estas entidades podem facilmente expor novos projetos, submeter propostas e envolver-se com um maior leque de fornecedores de produtos/ serviços.

1.1.1 Público-Alvo

- **Desenvolvedores de Software:** Desenvolvedores envolvidos na implementação do sistema devem-se focar nos requisitos funcionais e não funcionais. Estes leitores deverão entender os detalhes técnicos descritos nas secções que especificam as funcionalidades do sistema.
- **Gestores de Projeto:** Gestores de projeto devem concentrar-se nas secções de âmbito, cronogramas e prioridades para entender os requisitos de alto nível e a organização do projeto, assim como o impacto de cada funcionalidade no planeamento do desenvolvimento.
- **Equipa de Garantia de Qualidade (QA):** A equipa de garantia de qualidade deve priorizar as secções de requisitos funcionais e não funcionais, especialmente as partes que descrevem os critérios de aceitação e condições para garantir que os testes estejam de acordo com as especificações.
- **Equipa de Marketing e Stakeholders:** Para membros de marketing ou das partes interessadas que precisam de uma visão geral, as secções iniciais, que descrevem os objetivos, o propósito do sistema e a visão geral das funcionalidades, são as mais relevantes.

1.1.2 Funcionalidades

A presente subsecção detalha as principais funcionalidades e características que o sistema oferece.

1. Entidades

- O sistema deve permitir a gestão de entidades coletivas e singulares (empresas e indivíduos).
- O sistema deve permitir vincular, remover e editar dados pessoais de entidades singulares associadas a entidades coletivas.
- O sistema deve permitir que uma entidade coletiva aceda e gira projetos requisitados por outras entidades caso seja atribuído a este bem como todos os funcionários associados a ele.

2. Projetos

- O sistema deve permitir que as entidades requisitem e publiquem projetos ou serviços.
- O sistema deve permitir que as entidades avaliem propostas recebidas para a execução de um projeto.
- O sistema deve permitir a gestão de projetos desde a sua publicação até à sua conclusão.
- O sistema deve possibilitar a negociação de propostas de forma direta entre entidades ou através da listagem de serviços.
- O sistema deve permitir a realização de uma pesquisa filtrada para facilitar a procura de serviços a realizar ou produtos requisitados.
- O sistema deve permitir avaliar todas as entidades (singulares ou coletivas) associadas a um projeto.

3. Propostas

- O sistema deve permitir que as entidades enviem e recebam propostas para projetos listados ou requisitados.
- O sistema deve permitir a gestão de propostas enviadas e recebidas, facilitando o acompanhamento durante o ciclo de vida do projeto.
- Deve ser possível a utilização de um "Chat" ao nível do projeto para facilitar a comunicação entre cliente e fornecedor.

4. Ficheiros

- O sistema deve permitir a importação de documentos e especificações necessárias para a realização dos serviços ou produtos requisitados.

5. Pagamentos

- O sistema deve permitir a gestão de pagamentos, oferecendo planos de assinatura que concedem acesso a diferentes funcionalidades do sistema.
- O sistema deve permitir a escolha de diferentes planos de assinatura.

1.1.3 Arquitetura do Sistema

- **Backend**
 - **ASP.NET CORE WebAPI** para o desenvolvimento da API; (Microsoft, ASP.NET Core WebAPI)
 - **Entity Framework Core** para ORM, facilitando a manipulação dos dados do sistema. (Microsoft, Entity Framework Core, 2021)
 - Implementação de **JWT (JSON Web Tokens)** para autenticação e autorização. (jwt.io, 2024)
- **Frontend**
 - **Flutter**. (Flutter, 2024) para a versão web.
 - **Ionic**. (Ionic, 2024) para ver mobile.
- **Documentação da API**
 - **Swagger**. (OpenAI, 2024)
- **Controlo de versões, e integração contínua**
 - **GitLab** para o repositório de código, com pipelines de CI/CD utilizando **GitLab CI/CD**
 - Automatizar a atividade de Deploy em ambientes Docker, para garantir que a API possa ser facilmente testada e lançada em produção. (Docker, 2024)
- **Containerização com Docker**
 - Utilizar Docker para garantir que o ambiente de desenvolvimento e produção seja consistente.

1.2 Convenções do Documento

Este Documento de Especificação de Requisitos de Software (SRS) segue as seguintes convenções e padrões tipográficos para garantir consistência e clareza na apresentação dos requisitos:

1.2.1 Fonte e Formatação

- O texto geral deste documento está formatado na fonte **Porto Sans**, tamanho 12.
- Títulos e subtítulos seguem uma hierarquia clara utilizando **negrito** e diferentes tamanhos para cada nível de secção:
 - **Título de Secção (Ex: 1.0)**: Porto Sans 18pt, negrito.
 - **Subsecção (Ex: 1.1, 1.2)**: Porto Sans 14pt, negrito.
 - **Sub-subsecção (Ex: 1.1.1, 1.2.1)**: Porto Sans 12pt, negrito.

1.2.2 Destaque de Palavras-Chave

- Palavras ou termos de especial importância são realçados em **itálico**.
- Componentes críticos ou definições específicas são indicados entre "**aspas**" para chamar a atenção especial.

1.2.3 Requisitos

- Cada requisito é identificado por um número exclusivo e sequencial, começando com "R" seguido por um dos identificadores "F", "NF" (funcional ou não-funcional) terminando com um número composto por 3 dígitos (Ex.: RF101, RNF103).
- As prioridades de requisitos são indicadas utilizando as seguintes marcações:
 - Obrigatório: Indica que o requisito é essencial para o funcionamento do sistema.
 - Desejável: Indica que o requisito pode ser implementado para melhorar a usabilidade, mas não é essencial.
 - Opcional: Requisitos que podem ser adicionados no futuro ou dependem de certas condições.

1.2.4 Referências Cruzadas

Para garantir uma fácil navegação, sempre que um requisito ou secção fizer referência a outra parte deste documento, a numeração da secção relevante será incluída (Ex.: ver secção 2.3).

1.3 Sugestões de Leitura

Este Documento de Especificação de Requisitos de Software (SRS) foi preparado para satisfazer as necessidades de diferentes grupos de leitores, cada um com seu próprio interesse e nível de envolvimento no projeto.

Abaixo estão descritos os tipos de leitores e as partes do documento mais relevantes para cada um, assim como uma sugestão de sequência para leitura.

1.3.1 Organização do Documento

Este SRS está organizado nas seguintes secções principais:

1. **Introdução:** Apresenta o objetivo do sistema, convenções utilizadas no documento, e âmbito do produto.
2. **Arquitetura** (Diagrama de Pacotes): Fornece uma visão geral dos componentes principais do sistema.
3. **Requisitos de Software:** Detalha os requisitos funcionais e não funcionais do sistema, descrevendo as funcionalidades esperadas.
4. **Diagramas UML:** Explica as interações e casos de uso das principais entidades do sistema.
5. **Product Backlog:** Contém uma lista detalhada de funcionalidades e tarefas organizadas de forma priorizada.

6. Documentação da instância do GitLab: Explica a organização dos Boards e Labels usados para gerir o trabalho.
7. Base de Dados: Inclui o desenho conceptual e o modelo lógico da base de dados.
8. Mockups: Apresenta os modelos visuais da interface, tanto web como mobile, que guiarão o desenvolvimento do Frontend.

1.4 Âmbito do Produto

Este documento especifica o software MoldMe, cuja finalidade é melhorar a comunicação entre empresas da indústria de moldes, facilitando a colaboração entre fornecedores e fabricantes através de uma plataforma centralizada. O produto permite que empresas publiquem projetos, recebam propostas de fornecedores e possibilita a gestão durante a execução de trabalhos de maneira transparente e eficiente.

1.4.1 Funcionalidades no Âmbito

Seguidamente estarão apresentadas as funcionalidades que estão compreendidas no âmbito do projeto e serão implementadas até à conclusão do mesmo.

Este software possibilitará (ver secção 1.1.1):

- Criar, Visualizar, Editar, Remover perfil da entidade (coletiva ou singular), possibilitar o vínculo a uma entidade coletiva (ponto 1).
- Submeter projetos na plataforma (ponto 2).
- Procurar projeto por filtro desejado (ponto 2).
- Após terminar projeto, avaliação mútua entre entidades associadas ao mesmo (ponto 2).
- Aceitar ou recusar propostas mesmo (ponto 2).
- Visualizar detalhes de projeto tais como documentação associada e entidades intervenientes mesmo (ponto 2).
- Assinalar entidade(s) singular(es) responsáveis por projeto mesmo (ponto 2).
- Comunicação entre entidades a nível do projeto e Suporte e nível do sistema/projeto. (ponto 4);
- Importação de ficheiros de especificação/documentação de serviços/produtos a nível do projeto (ponto 5);
- Gestão de pagamentos referentes à fidelização por subscrição, com possibilidade de acesso ao histórico de pagamentos (ponto 6);

1.4.2 Funcionalidades Fora de Âmbito

As seguintes funcionalidades encontram-se fora de âmbito do projeto, não sendo incluídas durante o desenvolvimento, excluindo, portanto:

- Implementação de sistemas de ERP e CRM (por questões de controlo da quantidade do trabalho a realizar e da exigência do mesmo, o grupo de trabalho chegou ao consenso de não implementar os sistemas ERP e CRM e deixar esta funcionalidade para uma versão futura, fora do âmbito da unidade curricular).
- Sistema de descontos e ofertas, incluindo códigos promocionais (devido a questões de gestão de tempo e custo de implementação).
- Destaque de anúncios e pedidos de uma forma priorizada por pagamento (devido a questões de gestão de tempo e custo de implementação).
- Intervalo de preço médio para projeto (devido a falta de dataset que forneça dados precisos e atualizados).

1.4.3 Objetivos e Benefícios

- **Melhoria da Produtividade:** O sistema simplifica a exposição de novos projetos e a comunicação entre empresas, reduzindo o tempo necessário para encontrar fornecedores e fechar acordos.
- **Colaboração Transparente:** O produto garante que todas as partes envolvidas tenham acesso a documentos, avisos e atualizações, promovendo a transparência durante o ciclo de vida do projeto.
- **Sustentabilidade e Eficiência:** Ao conectar empresas diretamente numa plataforma unificada, o sistema reduz a necessidade de intermediários e custos associados aos mesmos, otimizando processos e promovendo uma cadeia de fornecimento mais sustentável.

1.5 Referências

Docker, I. (2024). *Docker*. Obtido de <https://docs.docker.com/>

Flutter. (2024). Obtido de <https://docs.flutter.dev/>

jwt.io. (2024). *JWT (JSON WEB Tokens)*. Obtido de <https://jwt.io/introduction>

Microsoft. (25 de maio de 2021). *Entity Framework Core*. Obtido de <https://learn.microsoft.com/en-us/ef/core/>

Microsoft. (s.d.). *ASP.NET Core WebAPI*.

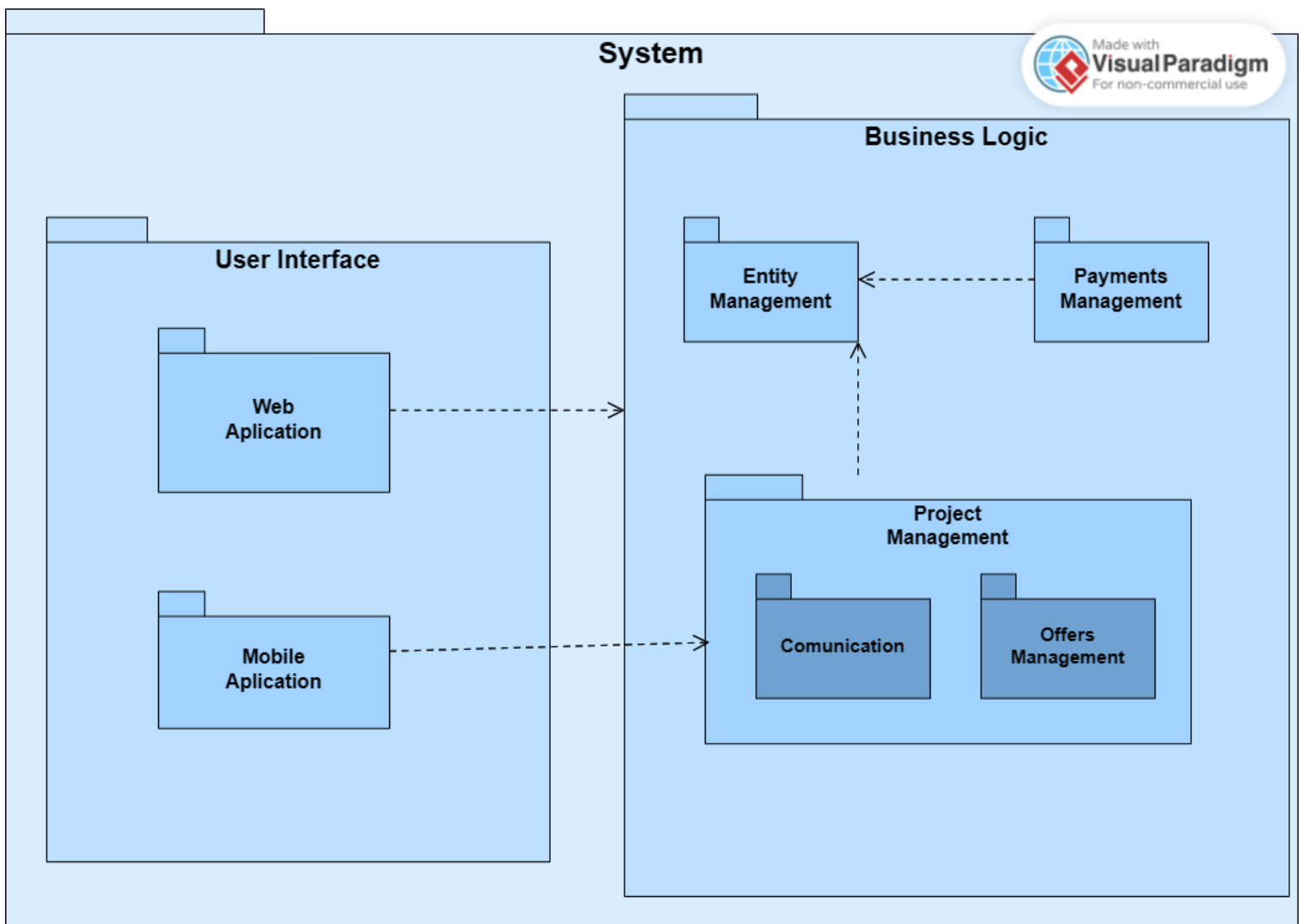
OpenAI. (2024). *Swagger*. Obtido de <https://swagger.io/specification/>

2. Arquiteturas

2.1 Alto Nível (Diagrama de Pacotes)

Um diagrama de pacotes é um tipo de diagrama estrutural em Unified Modeling Language (UML) que é usado para representar a organização e a comunicação de vários elementos dentro de um sistema. Esses elementos podem incluir classes, interfaces, componentes e outros pacotes.

Essencialmente, é uma maneira de agrupar elementos relacionados em "pacotes" para simplificar sistemas complexos, assim como as pastas ajudam a organizar arquivos num computador.



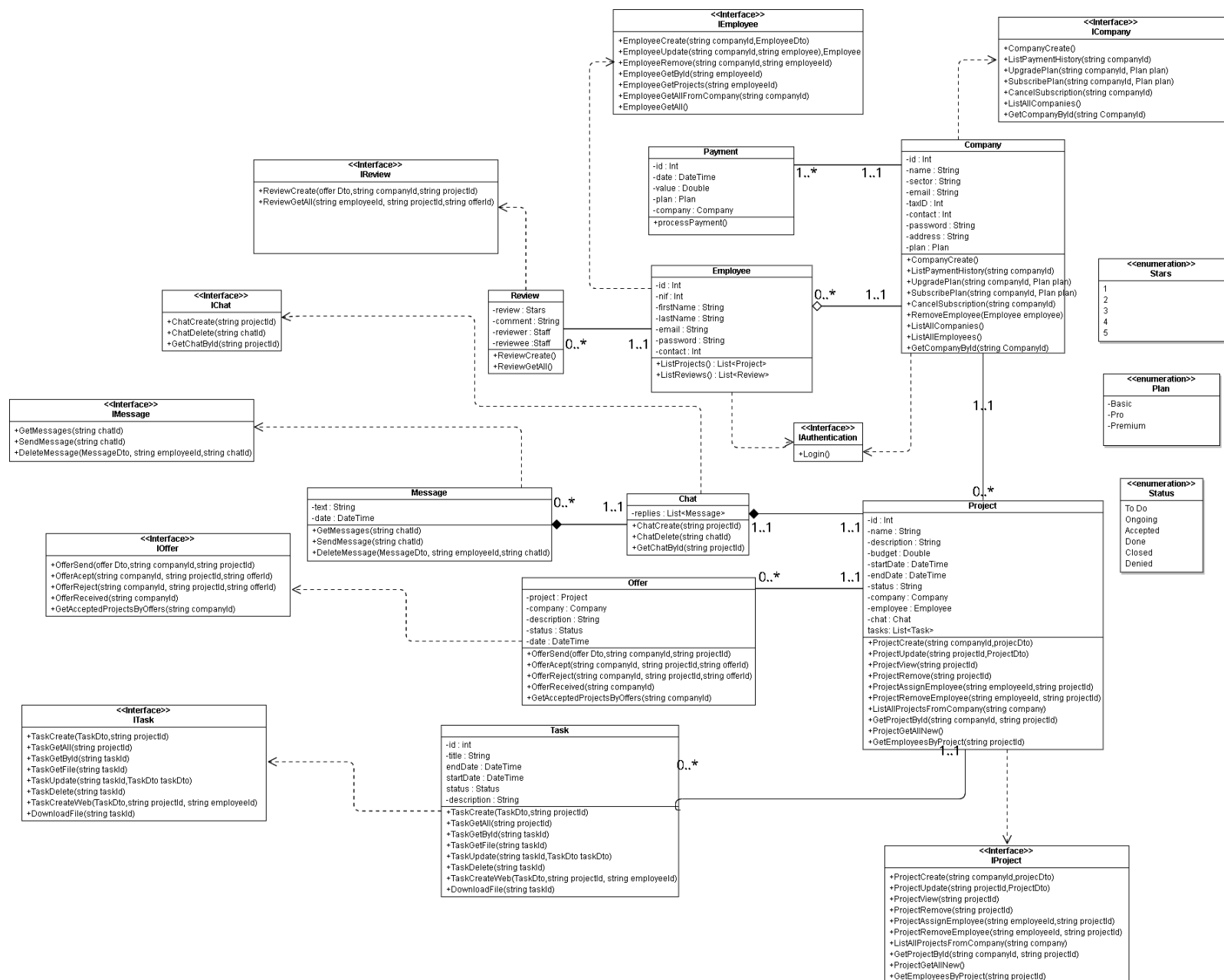
No prévio diagrama de pacotes é possível visualizar num todo a organização dos pacotes relacionados ao sistema. A lógica do sistema está dividida por três pacotes, são eles:

- User Interface
 - Dividida em Web Application e Mobile Application pois o software terá suporte para mobile e web, podendo os utilizadores aceder à mesma a partir do browser do seu computador ou diretamente a partir do telemóvel.
- Business Logic
 - Este pacote centraliza a lógica de negócios e está subdivido em três subsistemas principais.
 - O project Management, que abrange a gestão de projetos e contem duas áreas associadas, a Communication, que serve para a comunicação relacionada ao projeto e a Offers Management que serve para gerir as propostas dentro do projeto.
 - Para além destes aspetos também subentende que dentro deste pacote também haja a gestão de entidades que está depende do processo de pagamentos.
- Data Management
 - Este pacote compreende a camada onde os dados são processados, alterados, guardados e acedidos.

2.2 Diagrama de Classes

Um diagrama de classes em Unified Modeling Language (UML) é um tipo de diagrama de estrutura estática que descreve a estrutura de um sistema, mostrando as classes do sistema, atributos, operações e as relações entre os objetos.

No diagrama de classes acima está representada a organização da implementação do sistema.



3. Requisitos de Software

O sistema será organizado de modo a atender a determinados requisitos, sendo estes os funcionais e os não funcionais apresentados nos respetivos seguintes setores do documento, que definem assim como o sistema será construído e organizado, bem como as suas principais interações.

3.1 Requisitos Funcionais

No contexto de desenvolvimento de software, os requisitos funcionais são descrições das funcionalidades e comportamentos que um sistema deve proporcionar aos seus utilizadores.

Os requisitos funcionais, indicam o que sistema deve ser capaz de resolver problemas específicos e assim atender às necessidades dos utilizadores. O cumprimento destes requisitos é fundamental para garantir que o sistema atenda aos objetivos para qual o projeto foi criado.

Em seguida, estão apresentados todos os requisitos funcionais do projeto.

RF-101: Registar entidade

Descrição	O sistema deve permitir registar uma entidade.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas
Restrições	O utilizador deve fornecer dados obrigatórios como nome, email, número de identificação fiscal (NIF), durante o registo. O sistema deve verificar se o email já está registado. O registo apenas é concluído após a validação do email através de um link enviado para o utilizador.
Critérios de aceitação	O sistema deve garantir que os campos obrigatórios são validados antes da submissão do registo. O utilizador deve receber um email de confirmação para validar o seu registo. O utilizador só poderá aceder ao sistema após a validação do email.
Fluxo Principal	O utilizador acede à página de registo. O utilizador preenche os campos obrigatórios (nome, email, NIF, password). O sistema valida os dados introduzidos e envia um email de confirmação. O utilizador clica no link de confirmação enviado por email. O sistema finaliza o registo e notifica o utilizador de que o processo foi concluído com sucesso.

Fluxo Secundário	Caso o email já esteja registado, o sistema deve apresentar uma mensagem de erro e sugerir a recuperação de conta. Se a validação do email falhar, o utilizador deve ter a opção de reenviar o email de confirmação.
------------------	--

Tabela 1 – RF 101 – Registar entidade

RF-102: Autenticar entidade	
Descrição	O sistema deve permitir à entidade fazer login.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas Entidades singulares
Restrições	O sistema deve verificar a autenticidade do utilizador através das credenciais (email e password). Deve ser usada uma ligação segura (HTTPS) durante o processo de autenticação. Deve existir a opção de recuperação de password em caso de esquecimento.
Critérios de aceitação	O utilizador pode realizar o login com credenciais válidas. Caso as credenciais estejam incorretas, o sistema deve notificar o utilizador. O sistema deve permitir a recuperação da password via email.
Fluxo Principal	O utilizador acede à página de login. O utilizador introduz o email e password. O sistema valida as credenciais fornecidas. Se as credenciais forem válidas, o utilizador é autenticado e redirecionado para a sua página inicial.
Fluxo Secundário	Se as credenciais estiverem incorretas, o sistema exibe uma mensagem de erro. Se o utilizador esquecer a password, pode clicar na opção de recuperação de password para receber instruções por email

Tabela 2 – RF 102- Autenticar entidade

RF-103: Selecionar diferentes planos de assinatura	
Descrição	O sistema deve permitir a escolha de diferentes planos de assinatura.

Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas
Restrições	O sistema deve garantir que os planos de assinatura sejam apresentados de forma clara e compreensível. Apenas utilizadores com conta registada podem selecionar planos.
Critérios de aceitação	O sistema deve listar todos os planos de assinatura disponíveis. O utilizador deve ser capaz de visualizar detalhes sobre cada plano, incluindo preço e funcionalidades. O sistema deve confirmar a seleção do plano e atualizar as permissões do utilizador.
Fluxo Principal	O utilizador faz login no sistema. O utilizador navega até a seção de planos de assinatura. O sistema exibe todos os planos disponíveis. O utilizador seleciona um plano desejado. O sistema exibe os detalhes do plano escolhido e solicita confirmação. O utilizador confirma a seleção. O sistema atualiza as permissões do utilizador e notifica-o sobre a alteração.
Fluxo Secundário	Se o utilizador não confirmar a seleção, o sistema retorna à lista de planos disponíveis. Se ocorrer um erro ao processar a seleção, o sistema exibe uma mensagem de erro e solicita nova tentativa.

Tabela 3 – RF 103 – Selecionar diferentes planos de assinatura

RF-104: Gerir pagamentos

Descrição	O sistema deve permitir a gestão de pagamentos, oferecendo planos de assinatura que concedem acesso a diferentes funcionalidades do sistema.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas
Restrições	O sistema deve garantir que apenas utilizadores com conta registada possam gerir os pagamentos.

	Os planos de assinatura devem ser claramente definidos e apresentados aos utilizadores.
Critérios de aceitação	O sistema deve gerar notificações de pagamento bem-sucedido ou falhado. O utilizador deve conseguir visualizar o histórico de pagamentos.
Fluxo Principal	O utilizador faz o login no sistema. O utilizador navega até a área de gestão de pagamentos. O sistema apresenta os planos disponíveis e permite que o utilizador escolha. O sistema processa o pagamento e notifica o utilizador sobre o sucesso ou falha da transação. O sistema atualiza o acesso do utilizador às funcionalidades com base no plano selecionado.
Fluxo Secundário	Se o pagamento falhar, o sistema exibe uma mensagem de erro e solicita uma nova tentativa. O sistema deve permitir que o utilizador entre em contato com o suporte para resolver problemas de pagamento.

Tabela 4 - RF 104 - Gerir pagamentos

RF-105: Adicionar, editar, remover e vincular dados das entidades	
Descrição	O sistema deve permitir a gestão de entidades coletivas e singulares (empresas e indivíduos).
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas / gestor
Restrições	Autenticação de utilizadores/entidades para ter acesso à página de gestão. Apenas administradores/gestores podem adicionar ou remover entidades.
Critérios de aceitação	Notificação de erro caso haja algum erro durante as operações. Apenas administradores podem remover ou adicionar qualquer entidade associada.
Fluxo Principal	O utilizador acede à área de login do sistema. O sistema verificar se os dados introduzidos estão corretos, caso contrário será notificado que introduziu os dados errados. O utilizador tem acesso à página onde pode fazer a gestão dos dados.

	Após a edição dos dados, o sistema informa que os dados foram editados com sucesso, caso contrário avisa que ocorreu um erro.
Fluxo Secundário	O utilizador insere credenciais inválidas.

Tabela 5 – RF 105 – Adicionar, editar, remover e vincular dados das entidades

RF-106: Avaliar entidades	
Descrição	O sistema deve permitir avaliar todas as entidades (coletivas) associadas a um projeto.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas
Restrições	Apenas entidades envolvidas num projeto podem ser avaliadas. O sistema deve garantir que a avaliação só pode ser feita após a conclusão de um marco ou entrega.
Critérios de aceitação	O autor consegue visualizar todas as entidades associadas ao projeto para avaliação. O sistema permite que o autor faça uma avaliação de cada entidade.
Fluxo Principal	O utilizador faz o login no sistema. O utilizador acede à lista de projetos em que está envolvido. O sistema exibe as entidades associadas ao projeto. O utilizador seleciona a entidade a avaliar. O utilizador preenche e submete a avaliação.
Fluxo Secundário	Se houver problemas técnicos ao submeter a avaliação, o sistema exibe uma mensagem de erro e solicita nova tentativa.

Tabela 6 – RF 106 – Avaliar Entidades

RF-201: Publicar um projeto ou serviço	
Descrição	O sistema deve permitir que as entidades publiquem projetos ou serviços.
Prioridade	Urgente SIM Importante SIM

Atores	Entidades coletivas / gestor
Restrições	O utilizador deve estar autenticado para publicar um projeto ou serviço. Todos os campos obrigatórios devem ser preenchidos antes da publicação. O projeto ou serviço deve ser validado pelo sistema antes de ser publicado.
Critérios de aceitação	O sistema confirma a publicação do projeto ou serviço com sucesso. O sistema exibe uma mensagem de erro caso algum campo obrigatório não esteja preenchido. O projeto ou serviço fica visível para outros utilizadores após a publicação.
Fluxo Principal	O utilizador faz o login no sistema. O utilizador seleciona a opção de publicar um projeto ou serviço. O utilizador preenche os detalhes necessários (nome, descrição...). O sistema verifica se todos os campos obrigatórios estão preenchidos. O utilizador submete a publicação. O sistema valida a informação e confirma que foi publicado o projeto ou serviço.
Fluxo Secundário	Se algum campo obrigatório não estiver preenchido, o sistema exibe uma mensagem de erro e solicita a correção. Se houver um erro na validação do projeto ou serviço, o sistema informa o utilizador e impede a publicação.

Tabela 7 – RF 201 – Publicar um projeto ou serviço

RF-202: Editar, remover um projeto	
Descrição	O sistema deve permitir que as entidades editem ou removam projetos ou serviços.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas / gestor
Restrições	Apenas as entidades que criaram um projeto ou serviço podem editar ou remover o mesmo.
Critérios de aceitação	O sistema deve permitir a edição de todos os campos relevantes de um projeto ou serviço. O utilizador deve ser capaz de remover um projeto ou serviço.

Fluxo Principal	O utilizador faz o login no sistema. O utilizador acede à lista de projetos ou serviços. O utilizador seleciona um projeto ou serviço para editar ou remover. O sistema exibe as opções de edição ou remoção. O utilizador realiza as alterações ou escolhe remover. O sistema guarda as alterações ou remove o projeto/serviço e notifica o utilizador.
Fluxo Secundário	Se a edição ou remoção falhar, o sistema deve exibir uma mensagem de erro e solicitar nova tentativa.

Tabela 8 – RF 202 – Editar, remover um projeto

RF-203: Aceder a projetos requisitados por outras entidades	
Descrição	O sistema deve permitir que uma entidade coletiva aceda e gere projetos requisitados por outras entidades.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas
Restrições	Autenticação de entidades para ter acesso à página para gerar projetos. Apenas entidades autorizadas podem ver e gerar os projetos.
Critérios de aceitação	Entidades coletivas autenticadas podem aceder à lista de projetos requisitados. Entidades coletivas autenticadas podem gerar projetos requisitados. O sistema informa caso haja um erro ao gerar um projeto.
Fluxo Principal	A entidade coletiva faz login no sistema. O sistema valida as credenciais. Se as credenciais forem válidas, a entidade é redirecionada para a página de gestão de projetos. Se forem inválidas, uma mensagem de erro é exibida. A entidade coletiva visualiza os projetos requisitados por outras entidades. A entidade seleciona um projeto da lista e opta por gerar o projeto. O sistema mostra o projeto e exibe uma mensagem de confirmação ou erro, conforme aplicável.
Fluxo Secundário	A entidade tenta aceder à página de projetos sem estar autenticada. O sistema redireciona para a página de login.

	A entidade insere credenciais inválidas. O sistema exibe uma mensagem de erro, permitindo novas tentativas de login.
--	--

Tabela 9 – RF 203 – Aceder a projetos requisitados por outras entidades

RF-204: Importar de documentos e especificações	
Descrição	O sistema deve permitir a importação de documentos e especificações necessárias para a realização dos serviços ou produtos requisitados.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas Entidades singulares
Restrições	Apenas utilizadores autorizados podem importar documentos. O sistema deve validar o formato dos documentos antes da importação.
Critérios de aceitação	O sistema deve permitir a importação de arquivos em formatos especificados (por exemplo, PDF, DOCX). O sistema notifica o utilizador sobre o sucesso ou falha da importação. Os documentos importados devem ser associados automaticamente ao projeto ou serviço correspondente.
Fluxo Principal	O utilizador faz o login no sistema. O utilizador navega até a seção de importação de documentos. O utilizador seleciona o arquivo a ser importado. O sistema valida o formato e conteúdo do arquivo. O sistema regista a importação e associa o documento ao projeto ou <u>serviço</u> correspondente. O sistema notifica o utilizador sobre o sucesso ou falha da importação.

Fluxo Secundário	Se o formato do arquivo não for suportado, o sistema exibe uma mensagem de erro e solicita que o utilizador escolha outro arquivo. Se ocorrer um erro durante a importação, o sistema regista a falha e solicita uma nova tentativa.
------------------	--

Tabela 10 – RF 204 – Importar de documentos e especificações

RF-205: Enviar e receber propostas

Descrição	O sistema deve permitir que as entidades enviem e recebam propostas para projetos listados ou requisitados.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas Entidades singulares
Restrições	Apenas entidades registadas no sistema podem enviar ou receber propostas. O sistema deve garantir que as propostas sejam visíveis apenas ao autor do projeto e à entidade que as submete.
Critérios de aceitação	O autor consegue visualizar todas as propostas submetidas para o seu projeto. O sistema permite que o autor aprove ou rejeite as propostas recebidas. O sistema notifica as entidades sobre a aceitação ou rejeição das suas propostas.
Fluxo Principal	O utilizador faz o login no sistema. O utilizador acede à lista de projetos ou serviços para os quais pretende enviar uma proposta. O utilizador submete uma proposta detalhada para o projeto ou serviço selecionado. O autor do projeto visualiza a proposta submetida. O autor aprova ou rejeita a proposta. O sistema notifica o utilizador sobre a decisão do autor.
Fluxo Secundário	Se houver um erro técnico ao enviar ou receber a proposta, o sistema exibe uma mensagem de erro e solicita nova tentativa.

Tabela 11 – RF 205 – Enviar e receber propostas

RF-206: Avaliar propostas recebidas

Descrição	O sistema deve permitir que as entidades avaliem propostas recebidas para a execução de um projeto.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas Entidades singulares
Restrições	Apenas o autor do projeto ou serviço publicado pode avaliar as propostas. O sistema deve garantir que todas as propostas são visíveis ao autor, mas não antes do prazo de submissão das mesmas. O autor do projeto ou serviço pode aprovar ou rejeitar propostas.
Critérios de aceitação	O autor consegue visualizar todas as propostas submetidas para o seu projeto ou serviço. O sistema permite que o autor faça a avaliação de cada proposta. O sistema notifica os utilizadores que as propostas foram avaliadas. O autor pode deixar um comentário sobre as propostas rejeitadas ou aprovadas.
Fluxo Principal	O utilizador autentica-se no sistema. O utilizador acede à lista de projetos ou serviços publicados por ele. O utilizador seleciona um projeto ou serviço com propostas submetidas. O sistema exibe a lista de propostas recebidas para o projeto ou serviço. O utilizador visualiza e avalia cada proposta, podendo aprovar ou rejeitar. O sistema notifica os utilizadores sobre a avaliação das suas propostas. O sistema regista o histórico das avaliações realizadas.
Fluxo Secundário	Se o prazo de submissão ainda não tiver terminado, o sistema impede o acesso à visualização das propostas. Se houver problemas na submissão da avaliação, o sistema exibe uma mensagem de erro e solicita nova tentativa.

Tabela 12 – RF 206 – Avaliar propostas recebidas

RF-207: Pesquisar por projetos ou serviços

Descrição	O sistema deve permitir a realização de uma pesquisa filtrada para facilitar a procura de serviços a realizar ou produtos requisitados.
Prioridade	Urgente SIM Importante SIM

Atores	Entidades coletivas Entidades singulares
Restrições	A pesquisa deve limitar-se a categorias pré-definidas.
Critérios de aceitação	O sistema apresenta resultados relevantes com base nos filtros aplicados. O sistema responde à pesquisa num tempo inferior a X segundos. O utilizador pode ver e filtrar os resultados de acordo com critérios disponíveis.
Fluxo Principal	O utilizador faz o login no sistema. O utilizador acede à funcionalidade de pesquisa. O sistema exibe as opções de filtragem disponíveis (ex.: tipo de serviço, localização). O utilizador escolhe os filtros desejados e submete a pesquisa. O sistema apresenta os resultados da pesquisa de acordo com os filtros aplicados.
Fluxo Secundário	Se o utilizador realizar a pesquisa sem aplicar filtros, o sistema apresenta todos os resultados disponíveis. Se houver um erro na pesquisa (ex.: falha na rede), o sistema exibe uma mensagem de erro e solicita nova tentativa.

Tabela 13 – RF 207 – Pesquisar por projetos ou serviços

RF-301: Negociar de forma direta as propostas recebidas	
Descrição	O sistema deve possibilitar a negociação de propostas de forma direta entre entidades.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas Entidades singulares
Restrições	O sistema deve garantir que apenas as entidades que submeteram e receberam propostas podem iniciar negociações. O histórico de negociação deve ser gravado para referência futura. Limites de tempo para as negociações podem ser impostos de acordo com as configurações do projeto.
Critérios de aceitação	O histórico da negociação é acessível para ambas as partes e protegido contra alterações.

	O sistema fornece uma notificação de encerramento de negociação com as condições acordadas, se aplicável. A negociação pode ser marcada como bem-sucedida ou falhada, e ambas as partes são notificadas do resultado.
Fluxo Principal	O utilizador efetua o login no sistema. O utilizador acede à lista de projetos ou serviços em que possui propostas recebidas ou submetidas. O utilizador seleciona a proposta desejada para iniciar uma negociação. O sistema abre um canal de comunicação direta entre as duas partes envolvidas. As duas partes discutem e ajustam as condições da proposta. O utilizador finaliza a negociação (bem-sucedida ou falhada), e o sistema regista o acordo. O sistema notifica ambas as partes sobre o encerramento da negociação.
Fluxo Secundário	Se uma das partes não responder dentro de um prazo estabelecido, a negociação é encerrada automaticamente. O utilizador pode cancelar a negociação a qualquer momento, o que encerra o processo de forma unilateral.

Tabela 14 – RF 301 – Negociar de forma direta as propostas recebidas

RF-302: Adicionar tarefa	
Descrição	O sistema deve permitir que os utilizadores adicionem tarefas associadas a um projeto ou serviço.
Prioridade	Urgente SIM Importante SIM
Atores	Entidades coletivas Entidades singulares
Restrições	O sistema deve permitir adicionar uma tarefa com data válida.
Critérios de aceitação	O utilizador pode adicionar uma nova tarefa com informações mínimas (título, descrição, prazo).
Fluxo Principal	O utilizador faz o login no sistema. O utilizador acede à página do projeto e visualiza a lista de tarefas atuais. O utilizador clica no botão "Adicionar Tarefa".

	O sistema apresenta um formulário para inserir os detalhes da tarefa (título, descrição, prazo). O utilizador preenche os campos obrigatórios e submete o formulário. O sistema valida os dados. A tarefa é criada e aparece imediatamente na lista de tarefas da página principal.
Fluxo Secundário	Caso o utilizador não preencha os campos necessários o sistema avisa e pede para continuar a submissão.

Tabela 15 – RF 302 – Adicionar tarefa

3.2 Requisitos Não Funcionais

Os requisitos não funcionais descrevem as restrições e qualidades que o sistema deve cumprir, estes requisitos não se referem a funcionalidades específicas, mas são essenciais para garantir a eficiência e a experiência do utilizador.

RNF-101: Suporte multiplataforma	
Descrição	O sistema deve ser compatível com múltiplas plataformas, permitindo acesso e utilização em web e mobile
Prioridade	Urgente – NAO Importante – SIM
Atores	Entidades Singulares Entidades Coletivas
Restrições	O sistema deve ser responsivo, garantindo que funciona em web e mobile.
Critérios de aceitação	O sistema adapta corretamente o tamanho do ecrã e funcionalidades de cada dispositivo A experiência do utilizador é consistente em todas as plataformas.
Fluxo Principal	O utilizador acede ao sistema através de PC, Web ou Mobile. O sistema adapta a interface ao formato do dispositivo utilizado.
Fluxo Secundário	

Tabela 16 – RNF 101 – Suporte multiplataforma

4. Diagramas UML

A linguagem de modelagem unificada (UML) foi criada com o objetivo de fornecer uma linguagem de modelação visual comum, isto com o intuito de facilitar o processo de arquitetura, design e implementação de sistemas de software. Estes diagramas, em geral descrevem os limites, a estrutura e o comportamento do sistema, bem como os objetivos envolvidos no sistema.

4.1 Casos de Uso

Os diagramas de casos de uso são uma ferramenta essencial na modelação de sistemas para que o utilizador possa compreender o comportamento de certas funcionalidades.

Este diagrama permite identificar a utilidade de um sistema tendo como principal objetivo a interação com os atores e os casos de uso.

Cada ator, que pode ser uma pessoa ou um sistema, desempenha um papel ao interagir com o sistema, e os casos de uso são as funcionalidades vistas pelos utilizadores. Durante o desenvolvimento do sistema, é essencial identificar os atores e respetivos casos de uso para compreender as interações principais e definir os principais cenários que ilustram as funcionalidades do sistema.

O diagrama que se segue, está relacionado com o processo de registo e autenticação das entidades. O utilizador pode começar por “Registar entidade”, processo que inclui introduzir os dados da empresa, selecionar um plano, efetuar o pagamento e por fim confirmar o email. Após que a entidade estiver registada no sistema, o utilizador poderá efetuar o login, após introduzir os dados, o sistema irá verificar se a password está correta.

O diagrama apresentado a seguir está relacionado com o processo de gestão de entidades dentro do sistema. Neste caso, o utilizador principal trata-se do “Gestor de entidade”, ou seja, um administrador que esteja responsável pela mesma.

O processo inicia-se com a possibilidade de gerir assinaturas, onde o gestor pode alterar o plano, seleccionar plano e efetua o pagamento ou então pode cancelar a assinatura. Outro caso de uso é a gestão de entidades, isto é o gestor pode adicionar novas entidades singulares (colaboradores) e atribuir as devidas permissões, tem a possibilidade de remover entidades e pode também editar os dados.

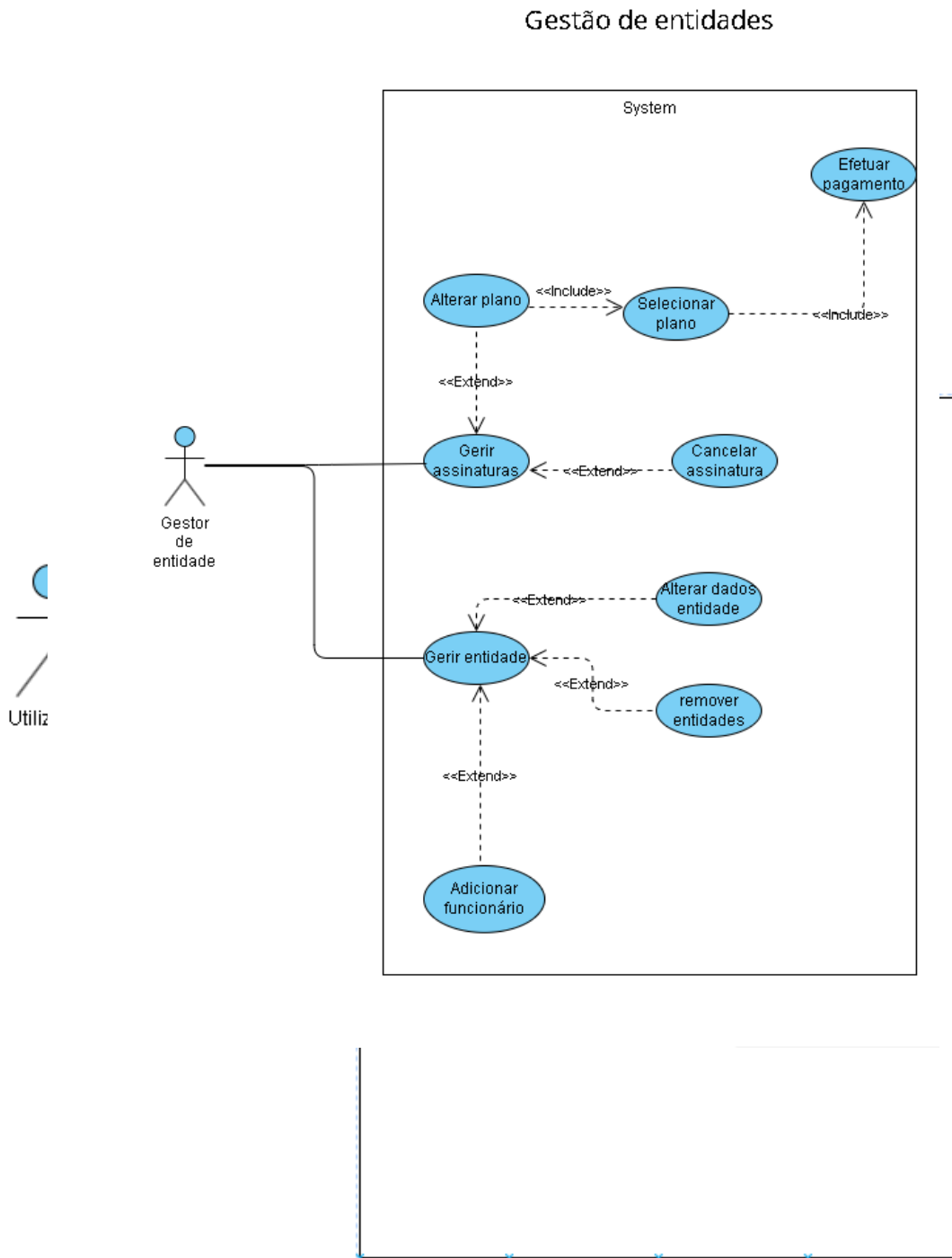


Figura 1 - Diagrama de Casos de Uso - Registo e autenticação.

Por fim, o diagrama apresentado, detalha o processo de gestão de projetos, onde existe dois atores principais, o Funcionário e Empresa. O funcionário tem a opção de pesquisar projetos no sistema, podendo filtrar ou ordenar projetos de modo a encontrar os projetos mais adequados.

O utilizador, após encontrar o projeto pretendido, pode abrir o mesmo e tem a opção de adicionar uma tarefa ou enviar uma mensagem para todos os colaboradores.

Por outro lado, a empresa tem acesso a todas estas atividades referidas anteriormente e para além disso, pode publicar um projeto e importar ficheiros para o mesmo, pode também visualizar o projeto e solicitar propostas e por fim pode também gerir o projeto que contem funcionalidades como avaliar as entidades envolvidas, remover o projeto e negociar as propostas com a finalidade de aceitar ou rejeitar a mesma.

Figura 2- Diagrama de Casos de Uso – Gestão de Entidades

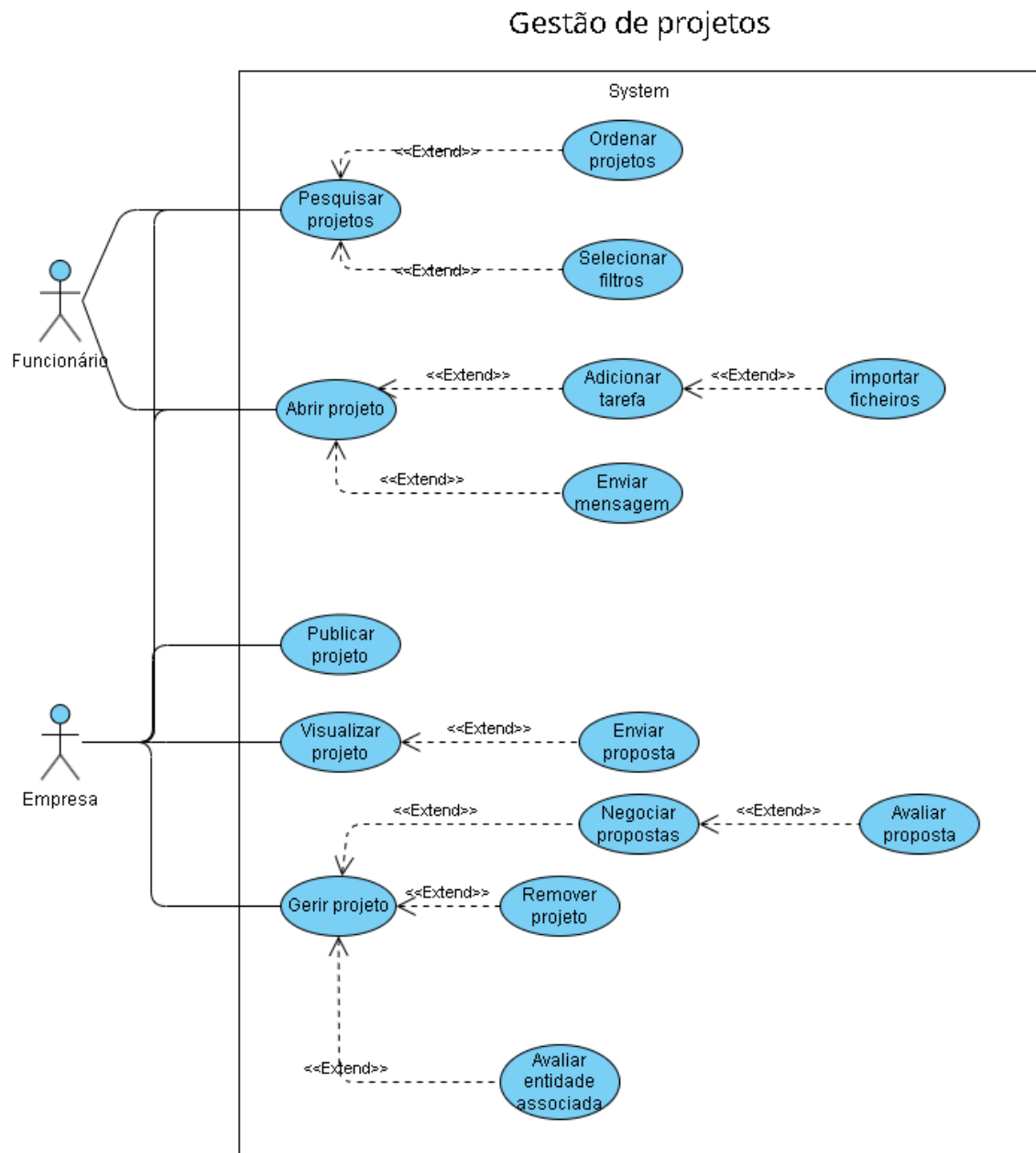


Figura 3 – Diagrama de Casos de Uso – Gestão de Projetos

5. Product Backlog

Histórico de revisão

Nome	Data	Razão para Alterações	Versão
João Pedro Teixeira & Miguel Ângelo Barbosa Ribeiro & Tiago Ribeiro & Leonel Carvalho	25/09/2024	Versão inicial (verificar versão submetido no milestone #3)	1.0
João Pedro Ferreira Teixeira & Miguel Ângelo Barbosa Ribeiro & Leonel Carvalho	5/11/2024	Priorização do product Backlog repensada	2.0
João Pedro Teixeira	24/11/2024	Atualização do Product Backlog e criação de Sprint Backlogs referentes ao frontend	3.0

O Product Backlog é uma lista priorizada e detalhada de trabalho para a equipa de desenvolvimento.

Os itens mais importantes são mostrados na parte superior do Product Backlog para que a equipa saiba o que entregar primeiro.

User Story	Descrição	Story Points
#1	Como empresa, quero iniciar, ver e concluir projetos, com intuito de gerir os meus projetos.	3
#2	Como empresa, quero gerir os membros de um projeto, para poder adicionar novos colaboradores	3
#3	Como membro de um projeto, quero criar tarefas, para todos os envolvidos saberem o que é que está a ser feito.	5
#4	Como membro de um projeto, quero poder enviar uma mensagem para a página do projeto, para poder perceber o estado atual do projeto ou solicitar algum tipo de informação.	3
#5	Como empresa, quero ver projetos abertos e enviar propostas, com intuito de solicitar o que for preciso para a minha empresa.	3
#6	Como empresa, quero gerir as propostas que foram feitas ao meu projeto, para poder decidir a melhor oferta ao meu projeto.	3

#7	Como utilizador, quero avaliar um colega com quem trabalhei num projeto de modo a criar um ambiente de trabalho transparente e saudável.	3
#8	Como empresa, quero adicionar, listar, remover e editar informações dos meus funcionários, para que possa adicionar novos funcionários ou remover algum que já não esteja na empresa.	5
#9	Como Staff de uma empresa, quero aceder aos meus projetos, para perceber quais atividades estou envolvido.	3
#10	Como empresa, quero ver e aceder os meus projetos, para poder ter acesso a tudo o que está relacionado ao meu projeto.	3
#11	Como utilizador, quero atualizar as minhas informações pessoais, com objetivo de manter a informação sempre atualizada.	3
#12	Como empresa, quero aceder ao meu histórico de pagamentos de assinatura de modo que possa gerir as minhas despesas com a plataforma.	3
#13	Como empresa, quero efetuar o meu registo na plataforma, para poder utilizar os serviços prestados.	5
#14	Como utilizador, quero fazer login na plataforma, para poder negociar os meus projetos.	5
#15	Como empresa na plataforma web, quero uma página onde possa criar um novo projeto, de forma a expor as minhas necessidades.	3
#16	Como empresa na plataforma web, quero uma página onde possa interagir com os meus projetos , pessoas envolvidas, tarefas e outros artefactos associados ao mesmo, com o intuito de visualizar o decorrer do mesmo, concluí-lo assim que seja necessário ou até mesmo aceitar ou rejeitar propostas e gerir acesso dos funcionários.	8
#17	Como membro de um projeto (plataformas móvel e web), quero um ecrã que contenha todo o conteúdo associado a um projeto.	3
#18	Como empresa na plataforma web, quero um ecrã onde possa poder gerir o acesso os meus funcionários, de forma a controlar o acesso à plataforma.	5
#19	Como membro de um projeto (plataformas web e móvel), quero um ecrã que me permita criar novas tarefas onde posso adicionar documentos e testamentos importantes para o projeto, com o intuito de manter um ambiente atualizado e transparente entre todos os membros do projeto.	5
#20	Como membro de um projeto (plataformas web e móvel), quero um ecrã que possibilite o envio e receção de mensagens que considero importantes para o chat do projeto, de forma a manter um ambiente atualizado e transparente entre todos os membros do projeto.	3

#21	Como empresa na plataforma web, quero um ecrã que me permita enviar uma proposta de serviço para um projeto e mais tarde aceder a esta proposta de forma a perceber se fui aceite para executar o mesmo.	3
#22	Como empresa na plataforma web, quero uma página onde possa aceder a todos os projetos a que me posso candidatar, de forma a fazer uma proposta ao mesmo.	3
#23	Como membro de um projeto(plataformas web e móvel), quero um ecrã onde possa avaliar um outro colega de um projeto, de forma a manter a transparência entre a plataforma.	5
#24	Como empresa na plataforma web, quero aceder a um ecrã onde me permita adicionar novas informações e editar as minhas informações privadas, para poder manter a plataforma atualizada com as minhas atualizações mais recentes.	3
#25	Como utilizador (plataforma web ou móvel), quero ter acesso a uma página que contenha um perfil público de uma empresa ou funcionário, de forma a conhecer os meus colegas de trabalho ou uma empresa com a qual possa vir a trabalhar.	3
#26	Como utilizador (plataforma web ou móvel), quero poder pesquisar por uma entidade, de forma a poder encontrar alguém ou uma empresa e aceder ao seu perfil público.	5
#27	Como empresa (plataforma web), quero aceder a um Pop-up do Paypal para pagar a assinatura contratada , de forma a manter-me ativo na plataforma.	8
#28	Como utilizador (plataforma web e móvel), quero ter acesso a um mapa visual com a localização geográfica de uma empresa, de forma a poder localizar a sede de forma mais fácil.	8
#29	Como utilizador (plataforma web ou móvel), quero ter acesso a filtros básicos de forma a facilitar a minha procura por projetos e/ou entidades.	3
#30	Como utilizador (plataforma web ou móvel), quero ter acesso à página de login e botão de logout, de forma a controlar o meu acesso à plataforma na plataforma .	3
#31	Como empresa (plataforma web), quero aceder a uma página de registo na plataforma, de forma a poder usufruir dos produtos que a MoldMe oferece.	3
#32	Como utilizador web, quero uma Homepage que fale sobre o compromisso do projeto MoldMe e possibilidade de registo, de forma a perceber se a sua solução satisfaz as minhas necessidades.	3

5.1 Sprint#1 Backlog

#1	Configuração da base de dados SQL + Regras Firewall + Modelação de classes	
----	--	--

5.2 Sprint#2 Backlog

User Story	Descrição	Story Points
#1	Como empresa, quero iniciar, ver e concluir projetos, com intuito de gerir os meus projetos.	3
#2	Como empresa, quero gerir os membros de um projeto, para poder adicionar novos colaboradores.	3
#3	Como membro de um projeto, quero criar tarefas, para todos os envolvidos saberem o que é que está a ser feito.	5
#4	Como membro de um projeto, quero poder enviar uma mensagem para a página do projeto, para poder perceber o estado atual do projeto ou solicitar algum tipo de informação.	3
#5	Como empresa, quero ver projetos abertos e enviar propostas , com intuito de solicitar o que for preciso para a minha empresa.	3
#6	Como empresa, quero gerir as propostas que foram feitas ao meu projeto, para poder decidir a melhor oferta ao meu projeto.	3
#7	Como utilizador, quero avaliar um colega com quem trabalhei num projeto de modo a criar um ambiente de trabalho transparente e saudável.	3
#8	Como empresa, quero adicionar, listar, remover e editar informações dos meus funcionários, para que possa fazer a gestão dos funcionários da empresa.	5
#9	Como Staff de uma empresa, quero aceder aos meus projetos, para perceber quais atividades estou envolvido.	3
#10	Como empresa, quero ver e aceder os meus projetos, para poder ter acesso a tudo o que está relacionado ao meu projeto.	3
#11	Como utilizador, quero poder aceder ao perfil público de outro utilizador, seja este funcionário ou uma empresa de modo a poder obter mais informações sobre esse indivíduo.	3
#15	Como empresa, quero aceder ao meu histórico de pagamentos de assinatura de modo que possa gerir as minhas despesas com a plataforma.	3
#16	Como empresa, quero efetuar o meu registo na plataforma, para poder utilizar os serviços prestados.	5
#17	Como utilizador, quero fazer login na plataforma, para poder negociar os meus projetos.	5

Sprint#3 Backlog

	Descrição	Story Points
#18	Correção ao backend e adicionar interfaces	5
#19	Como empresa na plataforma web, quero uma página onde possa criar um projeto, de forma a expor as minhas necessidades.	3
#20	Como empresa na plataforma web, quero uma página onde possa interagir com os meus projetos , pessoas envolvidas, tarefas e outros artefactos associados ao mesmo, com o intuito de visualizar o decorrer do mesmo, concluí-lo assim que seja necessário ou até mesmo aceitar ou rejeitar propostas e gerir acesso dos funcionários.	8
#21	Como membro de um projeto (plataformas móvel e web), quero um ecrã que contenha todo o conteúdo associado a um projeto.	3

5.3 Sprint#4 Backlog

	Descrição	Story Points
#22	Como empresa na plataforma web, quero um ecrã onde possa poder gerir o acesso os meus funcionários, de forma a controlar o acesso à plataforma.	5
#23	Como membro de um projeto (plataformas web e móvel), quero um ecrã que me permita criar novas tarefas onde posso adicionar documentos e testamentos importantes para o projeto, com o intuito de manter um ambiente atualizado e transparente entre todos os membros do projeto.	5
#24	Como membro de um projeto (plataformas web e móvel), quero um ecrã que possibilite o envio e receção de mensagens que considero importantes para o chat do projeto, de forma a manter um ambiente atualizado e transparente entre todos os membros do projeto.	3
#25	Como empresa na plataforma web, quero um ecrã onde possa poder gerir o acesso os meus funcionários, de forma a controlar o acesso à plataforma.	5
#26	Como membro de um projeto (plataformas web e móvel), quero um ecrã que me permita criar novas tarefas onde posso adicionar documentos e testamentos importantes para o projeto, com o	5

	intuito de manter um ambiente atualizado e transparente entre todos os membros do projeto.	
#27	Como membro de um projeto (plataformas web e móvel), quero um ecrã que possibilite o envio e receção de mensagens que considero importantes para o chat do projeto, de forma a manter um ambiente atualizado e transparente entre todos os membros do projeto.	3
#28	Como empresa na plataforma web, quero um ecrã que me permita enviar uma proposta de serviço para um projeto e mais tarde aceder a esta proposta de forma a perceber se fui aceite para executar o mesmo.	3

5.4 Sprint#5 Backlog

	Descrição	Story Points
#29	Como empresa na plataforma web, quero uma página onde possa aceder a todos os projetos a que me posso candidatar, de forma a fazer uma proposta ao mesmo.	3
#30	Como membro de um projeto(plataformas web e móvel), quero um ecrã onde possa avaliar um outro colega de um projeto, de forma a manter a transparência entre a plataforma.	5
#31	Como empresa na plataforma web, quero aceder a um ecrã onde me permita adicionar novas informações e editar as minhas informações privadas, para poder manter a plataforma atualizada com as minhas atualizações mais recentes.	3
#32	Como utilizador (plataforma web ou móvel), quero ter acesso a uma página que contenha um perfil público de uma empresa ou funcionário, de forma a conhecer os meus colegas de trabalho ou uma empresa com a qual possa vir a trabalhar.	3
#33	Como utilizador (plataforma web ou móvel), quero poder pesquisar por uma entidade, de forma a poder encontrar alguém ou uma empresa e aceder ao seu perfil público.	5
#34	Como empresa (plataforma web), quero aceder a um Pop-up do Paypal para pagar a assinatura contratada , de forma a manter-me ativo na plataforma.	8

6. Documentação da instância do GitLab

A documentação da instância do GitLab é uma referência fundamental para que haja uma gestão organizada ao longo do desenvolvimento do projeto. Esta secção abrange três áreas chave: as Issue Boards, os Labels e as User Stories.

6.1 User Stories

As User Stories descrevem as funcionalidades ou requisitos necessários. Estas User Stories são escritas em linguagem de negócio no seguinte formato padrão: Na qualidade de “papel/persona”, devo ser capaz de “requisito”, de forma a “benefício”.

6.2 Software Configuration Management Plan

O objetivo geral de um Plano de Gerenciamento de Configuração de Software (SCM) é documentar e informar as partes interessadas do projeto sobre o CM com o projeto, quais ferramentas de CM serão usadas e como serão aplicadas pelo projeto para promover o sucesso do mesmo. O SCM Plan define a estrutura e os métodos do projeto para:

- Identificar e definir itens uma Baseline para *Configuration Items* (CI)
- Controlar modificações de CIs
- Relatar e registar o status dos CIs e quaisquer modificações solicitadas
- Garantir a integridade, consistência e correção dos CIs
- Controlar a entrega dos CIs

6.2.1 Software Configuration Management

Esta secção é direcionada para a gestão de Software Configuration Management(SCM), centrando-se na definição da equipa SCM, as suas funções, responsabilidades e a forma como interagem com outros membros do projeto. O objetivo do SCM é assegurar que todas as alterações no software sejam geridas de forma organizada e controlada, minimizando riscos e maximizando a qualidade do produto.

No contexto deste projeto, a equipa SCM é responsável monitorizar, gerir e documentar as alterações ao código-fonte e a outros resultados chave do projeto. Esta secção irá detalhar ao pormenor as diferentes funções da equipa SCM e as suas interações com os membros que não pertencem à equipa SCM, como a equipa de desenvolvimento e o cliente.

6.2.1.1 Dev Team

A equipa de desenvolvimento interage com a equipa SCM para solicitar alterações ao código-fonte, atualizar versões e implementar correções.

Submetem pedidos de alteração (Change Requests) para aprovação pela equipa SCM antes da implementação.

Recebem feedback do SCM sobre a viabilidade e o impacto das alterações propostas.

6.2.1.2 Clientes

Os clientes podem ser envolvidos na fase de aprovação de alterações importantes, especialmente no caso de afetarem funções críticas ou prazos importantes.

6.2.2 Papeis SCM e Responsabilidades

Equipa responsável pela manutenção de toda a documentação oficial do projeto, código e software subjacente no decorrer do projeto.

[@8170538](#) – Miguel Ângelo Barbosa Ribeiro

[@8200489](#) – João Pedro Ferreira Teixeira

[@8210127](#) – Leonel da Silva de Carvalho

[@8210136](#) – Tiago André Cardoso Ribeiro

6.2.2.1 CM Leader

O CM leader é o [@8210136](#).

A responsabilidade do CM Leader é:

- elaborar o SCM Plan;
- supervisionar o CCB (Configura o Control Board);
- informar os membros da equipa sobre as suas responsabilidades;
- organizar também reuniões para efetuar a revisão do código dos membros e mantê-los focados nas tarefas e dentro do prazo.

6.2.2.2 Configuration Control Board (CCB)

A responsabilidade do CCB é aprovar todas as alterações do software e documentação durante o tempo do projeto. É também responsabilidade do CCB orientar os desenvolvedores sobre como eles tem impacto na aprovação de alterações.

6.2.2.3 Líder CCB

O Leader do CCB é o [@8210127](#) ao qual tem as seguintes responsabilidades:

- liderar reuniões e comunicar aos membros da equipa avisos das reuniões;
- aprovar de forma autónoma Change Requests a serem implementados
- realizar reuniões do CCB para rever/aprovar Change Requests, pode interromper um Change request específico, e definir novas prioridades.

6.2.3 Membros do CCB

Um membro do CCB é qualquer pessoa na CCB Role (temporariamente ou permanentemente). Um membro deve ser eleito pelo @8210127 (CCB Leader) para manter os documentos do projeto e código.

Role	Responsável	Responsabilidade
Scrum Master	@8200489	Lidera e facilita o processo ágil de desenvolvimento do projeto. Interage com a Equipa de projeto, bem como com o cliente e gestão durante o projeto
Product Owner	@8170538	Responsável pelo projeto na definição, gestão e priorização das necessidades do produto.
Scrum Team	@8170538 @8210127 @8210136 @8200489	Têm autoridade para tomar as ações necessárias em prol de alcançar os objetivos de cada sprint de modo que o que se entrega está correto e que o software satisfaz as necessidades dos clientes.

Legenda de Responsáveis

[@8170538](#) – Miguel Ângelo Barbosa Ribeiro

[@8200489](#) – João Pedro Ferreira Teixeira

[@8210127](#) – Leonel da Silva de Carvalho

[@8210136](#) – Tiago André Cardoso Ribeiro

6.3 Atividades CM

CM Activities são essenciais durante o ciclo de vida de um projeto, desde a fase de planeamento até ao seu encerramento. As CM Activities ajudam a manter a integridade dos produtos a serem desenvolvidos e evitam que sejam feitas alterações ao projeto sem qualquer forma de controlo. Seguidamente iremos abordar algumas CM Activities cruciais.

6.3.1 Configuration Identification

Para a realização e manutenção deste projeto será utilizado *“Semantic Versioning”* para a numeração de versões. São utilizados três números principais no formato de MAJOR.MINOR.PATCH.

Major: Indica mudanças que necessitam de atenção especial como grandes alterações ou revisões a funcionalidades. A cada sprint semanal este número deve ser incrementado.

Minor: Refere-se a adições de funcionalidades que são compatíveis com versões anteriores, que permitem atualizações sem problemas de falhas.

Patch: Representa correções de bugs ou melhorias internas que não afetam o bom funcionamento do projeto.

6.3.2 Peso das Issues

Normalmente, seguimos a escala de Fibonacci, que inclui os valores 1, 2, 3, 5, 8, 13, 21, para atribuir pesos às Issues. Iniciamos com o valor 1 e concluímos com o valor 8. Nesse contexto, a issue com peso 1 representa a menor carga de trabalho, enquanto a issue com peso 8 representa a maior carga de trabalho.

Cada número na escala reflete um aumento exponencial na complexidade e por consequente o trabalho associado a cada issue.

Essa prática ajuda a gerir melhor o tempo e os recursos necessários.

6.3.3 Identificação de Issues

As Issues, no momento da sua criação, são recomendadas a associação de uma ou mais Labels para que seja possível mitigar uma possível desorganização à medida que o projeto avança.

- [User Story]: Descrição de uma User Story
- [Change Request]: Comentário
- [Task Nº UserStory]: Exemplo de Comentário para Task desenvolvida pelos desenvolvedores.

6.3.3.1 Issue Boards

As Issue Boards permitem acompanhar visualmente o progresso de cada tarefa ao longo do ciclo de vida, podem estar organizadas de diferentes formas, como por exemplo em pedido de

alteração “Change Requests” ou desenvolvimento “Development”, estas divisões facilitam a gestão de atividades pendentes entre todos os membros da equipa.

- ✓ Backlog Boards
- ✓ Change Requests
- ✓ Development

6.3.4 Identificação de Labels

As Labels são utilizadas para classificar e priorizar as tarefas. Com base em diferentes categorias, com por exemplo: em progresso “In progress” ou uma nova funcionalidade “Feature”, com estas etiquetas associadas a cada tarefa a equipa consegue facilmente perceber o ponto da situação. Além disto, conta também com as Story Points que ajudam a estimar o esforço necessário para cada uma das tarefas.

✓ User Stories

- **Backlog** – O item está no Product Backlog, mas ainda não foi atribuído a um sprint.
- **Ready for Sprint** – Os itens são preparados e estão prontos para serem adicionados ao sprint.
- **In Progress** – Itens que estão a ser trabalhados ativamente.
- **Feature** – Novas funcionalidades a desenvolver.
- **Enhancement** – Melhoria dos recursos existentes.
- **Bug** – Problemas ou defeitos que precisam de ser corrigidos.

Story Points – Estimativas para indicar o esforço a aplicar em uma User Story do Product Backlog

- **SP 1** – Tarefa muito pequena, esforço mínimo.
- **SP 2** – Tarefa pequena com alguma complexidade.
- **SP 3** – Tarefa média, complexidade moderada.
- **SP 5** – Tarefa grande, alta complexidade.
- **SP 8** – Tarefa muito grande, complexidade muito alta
- **SP 13** – Tarefa de nível épico, complexidade extremamente alta
- **SP Unestimated** – Tarefa ainda não estimada ou esclarecida

✓ Development – Uma tarefa que envolve o desenvolvimento do trabalho programado.

- **To Do** – Tarefa está pronta, mas não iniciada.
- **In Progress** – Esta tarefa está atualmente em curso.
- **Code Review** – As tarefas são revistas em termos de qualidade ou melhoria.
- **Testing** – A tarefa está a ser submetida a um teste de controlo de qualidade.

- **Done** – A tarefa foi concluída e implementada.
- ✓ **Change Request** – Uma tarefa que envolve uma modificação no trabalho existente.
 - **Proposed** – Alterações solicitadas, mas ainda não consideradas.
 - **Approved** – As alterações propostas foram consideradas e aprovadas.
 - **In Progress** – Esta alteração está atualmente a ser implementada.
 - **Completed** – As alterações foram implementadas com êxito.
 - **Rejected** – Pedido de alteração analisado, mas rejeitado.
 - **High Priority** – Alterações com impactos significativos que exigem ação imediata.
 - **Medium Priority** – Mudança importante, mas não urgente.

UserStories (Product Backlog)

- UserStories Backlog 
- UserStories Ready for Sprint 
- UserStories In Progress 
- Feature 
- Enhancement 
- Bug 

Development (Workflow/Status)

- Dev To Do 
- Dev In Progress 
- Dev Code Review 
- Dev Testing 
- Dev Done 
- Priority High 
- Priority Medium 
- Priority Low 

Change Request

- Change Proposed 
- Change Approved 
- Change In Progress 
- Change Completed 
- Change Rejected 
- Change High Priority 
- Change Medium Priority 
- Change Low Priority 

Figura 4 – Identificação das labels

6.3.5 Configuration Control

Configuration Control implica que exista um processo formalizado para gerir alterações de forma controlada. Desta forma asseguramos que apenas alterações aprovadas são implementadas.

6.3.5.1 Configuration Control Process

1. Submit a Change Request in GitLab Issue Tracker
2. CCB reviews and decides
3. CCB assigns the issue
4. Check-out and switch to master branch
5. Pull
6. New Branch for the issue
7. Code
8. Test locally
9. Commit/Push
10. Request merge
11. Merge request approval
12. Merge
13. Push the merged master branch
14. Resolve the issue

6.3.6 Configuration Status and Accounting

- **Baselines**

ProductBacklog

Development

Change Request

- **ProductBacklog**

Baseline para gerir user stories definidas previamente no product backlog.

- **Development**

Baseline para gerir user stories definidas previamente em product backlog.

- **Change Request**

Baseline para gerir change requests da equipa de desenvolvimento.

- **Branches**

A definição de branches irá ser do seguinte formato "#-nome-issue".

Exemplo: 34-desenvolver-codigo

- **Testes de SW**

Os testes unitários identificados pelo modelo TC#. Sendo o identificador numérico inequívoco da issue (Exemplo: 5) e o um número sequencial para o teste daquela issue. Exemplos: TC503, TC1201.

6.4 CM no Ciclo de Vida do Desenvolvimento

6.4.1 Metodologia Scrum

O Scrum é uma metodologia ágil que se distingue pela alta adaptabilidade e pelo foco na entrega contínua de valor.

Composto por três fases principais que são o Pre game, Desenvolvimento e Post-Game, o Scrum permite que as equipas respondam rapidamente a alterações nos requisitos e que o produto evolua de forma contínua ao longo do ciclo de vida de desenvolvimento.

Ao contrário de metodologias tradicionais, como o Waterfall, que seguem uma sequência linear de etapas, o Scrum promove uma colaboração constante entre a equipa de desenvolvimento e as partes interessadas (stakeholders).

Reuniões regulares, como as Daily Scrum, Sprint Planning, Sprint Review e Sprint Retrospective, são essenciais para manter a comunicação, identificar e resolver problemas rapidamente, e ajustar a direção do projeto, conforme necessário, para melhorar a cada iteração.

6.4.2 Fases do Scrum

O Scrum é composto por três fases: **Pre-game**, **Desenvolvimento** e **Post-game**.

Na fase **Pre-game**, que inclui duas subfases – Planeamento e Arquitetura do Projeto, os requisitos são priorizados e é estimado o tempo necessário para a implementação. Durante o planeamento, ocorre a definição do sistema, a criação do Backlog do produto, e são estabelecidos elementos como a equipa do projeto e as ferramentas a utilizar. Todas as iterações são revistas pela equipa Scrum para definir compromissos. Na subfase de Arquitetura, que envolve o planeamento, são realizadas reuniões de revisão do projeto e a implementação dos itens do Backlog.

A fase de **Desenvolvimento** é a parte ágil do Scrum e é tratada como uma "Black Box". É observada através de várias práticas durante os sprints, sendo o Scrum responsável por um controlo constante. Cada sprint inclui fases tradicionais de desenvolvimento, tais como análise, definição de requisitos, projeto, evolução e entrega.

Na fase **Post-Game**, ocorre o encerramento das entregas. O sistema está pronto para ser entregue, e os requisitos estão concluídos, não havendo mais itens para acrescentar.

6.4.3 Scrum roles

- **Scrum Master:** Responsável por garantir que o projeto é executado de acordo com as práticas estabelecidas. O Scrum Master atua como um facilitador, removendo obstáculos que possam surgir durante o desenvolvimento, promovendo uma comunicação eficaz e garantindo a adesão aos princípios do Scrum.

- **Product Owner:** Responsável pela gestão e controle do projeto. O Product Owner tem a função de tornar visível a lista de itens do Backlog, definir prioridades e garantir que a equipa esteja focada nos requisitos mais importantes para o sucesso do projeto. É o representante dos interesses do cliente ou stakeholders.
- **Scrum Team:** A equipa de projeto que possui autoridade para tomar decisões necessárias com o objetivo de atingir as metas de cada sprint. É uma equipa auto-organizada e multifuncional, composta por profissionais que colaboram para entregar as funcionalidades prometidas no Backlog. A equipa Scrum trabalha de maneira colaborativa para atingir os objetivos estabelecidos para cada iteração.

6.4.4 Artefactos do Scrum

- **Product Backlog:** O Product Backlog é uma lista que define tudo o que é necessário no produto final. Esta lista inclui requisitos, funcionalidades e melhorias desejadas para o produto.
- O Product Backlog é dinâmico e evolui ao longo do tempo, sendo priorizado pelo Product Owner com base nas necessidades do cliente e nas mudanças no mercado. Este é um instrumento essencial para o planeamento e gestão do desenvolvimento do produto.
- **Estimativa de Esforço (Effort Estimation):** A Estimativa de Esforço é um processo iterativo no qual a equipa Scrum avalia o esforço necessário para concluir as atividades do Product Backlog. É um processo que se ajusta à medida que mais informações são disponibilizadas e à medida que a equipa ganha uma compreensão mais clara dos requisitos. A estimativa de esforço ajuda a equipa a planear melhor as iterações e a estabelecer metas realistas para o trabalho a ser realizado durante cada Sprint. Tanto o Product Owner como a equipa de Scrum têm responsabilidades neste processo colaborativo.

7. Evidências GitHub

Abaixo serão apresentadas evidências de utilização de CI/CD e metodologia SCRUM durante o projeto.

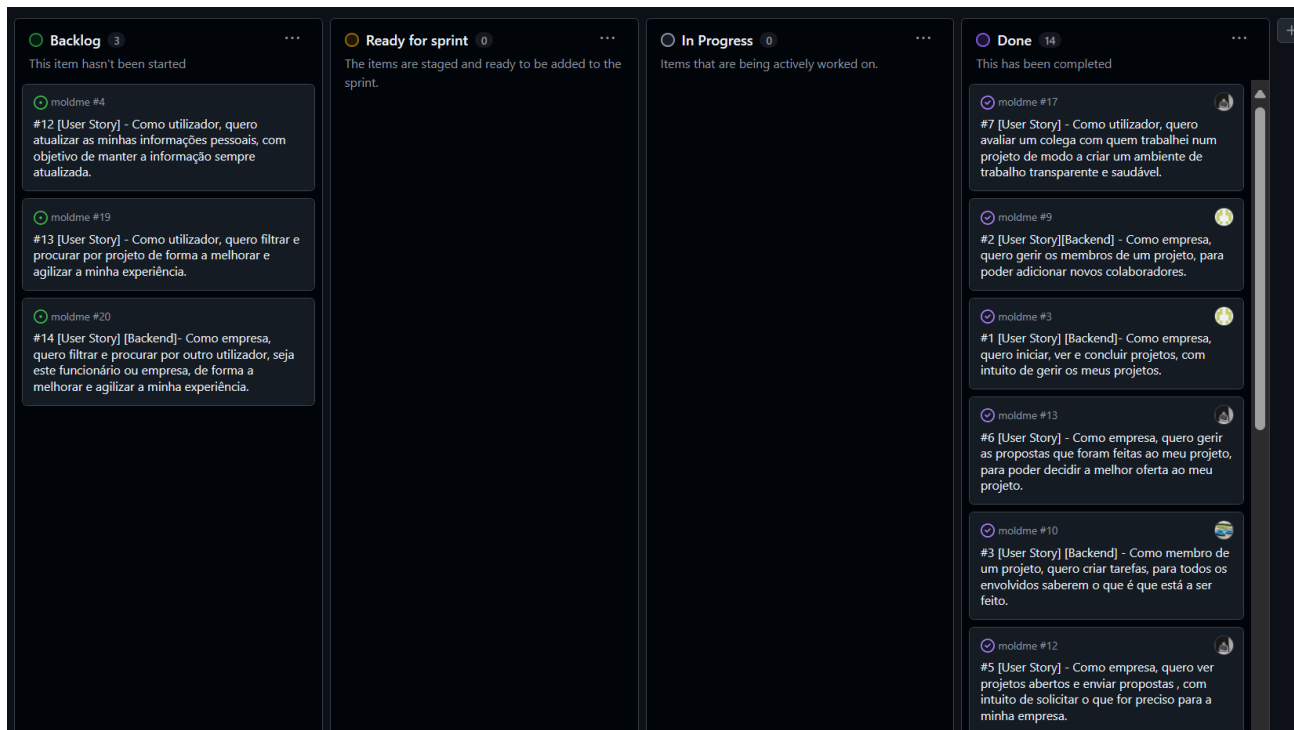


Figura 5 – Figura com a Board de Product Backlog- Evidência de uso do GitHub

Dev 44 change request adicionar dtos aos controllers #48

Merged pedroteixeira2002 merged 5 commits into dev from dev-44-change-request-adicionar-dtos-aos-controllers 2 days ago

Conversation 1 Commits 5 Checks 2 Files changed 27 +1911 -799

Leonel1001 commented 2 days ago

No description provided.

Leonel1001 added 5 commits 3 days ago

- DTOs
- Delete moldme.zip
- Todos os Dtos e controllers atualizados
- Correções finais aos testes
- Atualizar TODO

Leonel1001 added Priority: High ChangeRequest labels 2 days ago

Leonel1001 added this to the Sprint #3 milestone 2 days ago

Leonel1001 requested review from pedroteixeira2002, R1B3IR0 and TiagoRibeiro136 2 days ago

Leonel1001 self-assigned this 2 days ago

Leonel1001 linked an issue 2 days ago that may be closed by this pull request

[Change Request] - Adicionar DTOs aos controllers #44 Closed

pedroteixeira2002 commented 2 days ago

All code seems nice after team review!

pedroteixeira2002 merged commit b5f5ccb into dev 2 days ago

2 checks passed

View details Revert

Reviewers

- pedroteixeira2002
- R1B3IR0
- TiagoRibeiro136

Assignees

- Leonel1001

Labels

- ChangeRequest
- Priority: High

Projects

- Change Requests

Status: Completed

Milestone

Sprint #3

Development

Successfully merging this pull request may close these issues.

[Change Request] - Adicionar DTOs aos controllers

Notifications

Unsubscribe

2 participants

Lock conversation

Figura 6 - Exemplo de change request de uma correção que surgiu

All workflows

Showing runs from all workflows

162 workflow runs

Event ▾Status ▾Branch ▾Actor ▾

✓

Merge pull request #57 from R1B3IR0/new_testing

dev

Build & Test #162: Commit d7e9162 pushed by R1B3IR0

8 minutes ago

40s

...

✓

New testing

new_testing

Build & Test #161: Pull request #57 synchronize by R1B3IR0

9 minutes ago

40s

...

✓

Definição da validade do token

dev

Build & Test #160: Commit 557978e pushed by R1B3IR0

8 hours ago

53s

...

✗

Testing to 95% coverage

47-change-request-adicionar...

Build & Test #159: Pull request #56 synchronize by pedroteixeira2002

yesterday

36s

...

✓

update endpoint addProject

dev

Build & Test #158: Commit 71788b8 pushed by R1B3IR0

yesterday

47s

...

✓

Update Jwt Authorization

dev

Build & Test #157: Commit aa8e46b pushed by R1B3IR0

yesterday

47s

...

✓

Merge pull request #55 from R1B3IR0/53-melhorias-nas-classes-task-mes...

dev

Build & Test #156: Commit 2b6b9d5 pushed by R1B3IR0

yesterday

50s

...

✓

Testing and Upgrading Classes

53-melhorias-nas-classes-ta...

Build & Test #155: Pull request #55 opened by pedroteixeira2002

yesterday

42s

...

✗

Merge pull request #54 from R1B3IR0/52-change-request-melhorias-de-có...

dev

Build & Test #154: Commit 779eca3 pushed by pedroteixeira2002

2 days ago

31s

...

✗

Melhorias:

52-change-request-melhorias...

Build & Test #153: Pull request #54 opened by R1B3IR0

2 days ago

40s

...

✓

Merge pull request #51 from R1B3IR0/45--change-request--reforo-da-au...

dev

Build & Test #152: Commit 09dca49 pushed by TiagoRibeiro136

2 days ago

49s

...

Figura 7 - Evidências de CI/CD – Pipelines

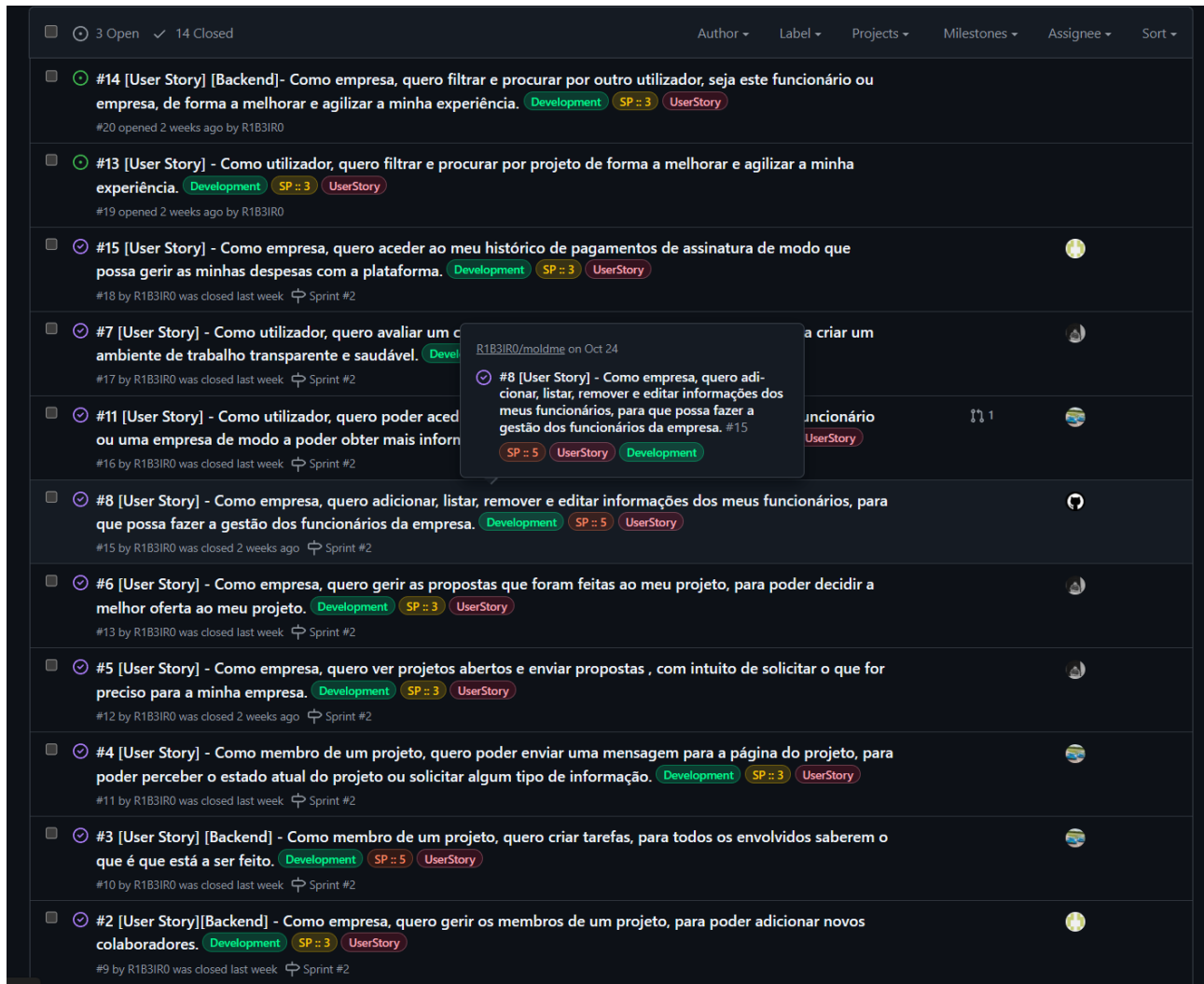


Figura 8 - Issues - UserStories

8. Base de Dados

A base de dados implementa todas as funcionalidades e relacionamentos necessários para suportar as principais funcionalidades do sistema. A mesma foi projetada com o objetivo de garantir a eficiência e integridade no armazenamento e gestão dos dados.

Desta forma o Microsoft SQL Server foi escolhido como plataforma de gestão de base de dados, o que oferece o suporte necessário para o armazenamento, consulta e manipulação de dados.

8.1 Desenho Conceptual

De forma a construir o desenho conceptual da base de dados de forma correta, os seguintes passos deverão ser executados pela seguinte ordem:

1. Identificar os tipos de entidade
2. Identificar os tipos de relacionamento
3. Identificar e associar atributos a tipos de entidade ou relacionamento
4. Determinar o domínio dos atributos
5. Determinar as chaves candidatas, primárias e alternativas
6. Verificar o modelo para redundância
7. Validar o modelo de dados conceptual em relação às transações do utilizador
8. Revisão do modelo de dados conceptual com o utilizador

8.1.1 Identificação de Tipo de Entidades

As entidades são um conceito fundamental na conceção e gestão de bases de dados. Existem vários tipos de entidades. São blocos de construção de bases de dados em todo o mundo, desde catálogos de bibliotecas a listas de correio e inventários de envio. Para a construção correta da base de dados do nosso caso de estudo, é essencial entender o que são entidades e como usá-las para criar uma base de dados. Nos seguintes passos iremos compreender os diferentes tipos de entidades e como registá-las num diagrama entidade-relacionamento.

Entidade	Descrição	Aliases	Ocorrências
Project	Representa um projeto introduzido por uma empresa.	Projeto	Um projeto pode ter zero ou mais propostas e tarefas associadas, e pode ter um ou mais funcionários e empresas associados.
Company	Representa uma entidade coletiva, cliente ou fornecedora.	Empresa	Cada empresa pode conter zero ou mais projetos e pode ter vários funcionários
Staff	Um indivíduo que trabalha para uma empresa.	Staff	Um funcionário pode estar associado a um ou mais projetos e realizar nenhuma ou uma ou mais tarefas.
Offer	Contém os detalhes de uma oferta ou negociação do projeto.	Proposta, Oferta	Um projeto pode receber nenhuma, uma ou mais propostas de diferentes empresas. Cada proposta é associada a um projeto específico.
Task	Unidade de trabalho dentro do projeto	Atividade, Trabalho, Tarefa	Cada projeto e funcionário contém zero ou mais tarefas.

Chat	Representa um Chat de um projeto, onde todos os colaboradores do mesmo podem enviar mensagens de texto.	Conversa	Cada projeto contém um e um só Chat.
Message	Representa uma mensagem de texto inserida num chat de um projeto	Mensagem	Um Chat pode conter um número indeterminado de mensagens. Apenas alguém com acesso ao chat poderá enviar mensagens.
File	Ficheiros que podem ser adicionados a uma Task.	Ficheiro, Documento	Uma Tarefa poderá ter zero ou mais ficheiros associados.
Review	Uma avaliação feita a alguém	Avaliação	Depois do término do projeto, todos os elementos podem (não obrigatório) avaliar todos os elementos com quem trabalharam. Não é possível avaliar-se a si mesmo.
Payment	Um pagamento feito pela empresa à plataforma	Pagamento	Dependendo da regularidade definida, uma empresa paga uma quantia definida à plataforma.

Tabela 6 – Entidades identificadas

8.1.2 Identificar os tipos de relacionamentos

As várias entidades podem e vão relacionar-se entre si de forma a criar uma base de dados concisa. A identificação de relacionamentos entre entidades em BDs é um dos conceitos fundamentais no design e uso de BDs. Em BDs, estes relacionamentos são expressos por meio de tabelas e chaves. No nosso caso de estudo, identificamos os seguintes relacionamentos entre as entidades acima identificadas

Entidade	Multiplicidade	Relação	Multiplicidade	Entidade
Project	1..*	Possui	0..*	Staff
	1..1	Contém	0..*	Task
	1..1	Possui	0..*	Offer
	1..1	Possui	1..1	Chat
Company	1..1	Realiza	0..*	Offer
	1..1	Emprega	1..*	Staff
	1..1	Lança	0..*	Project
	1..1	Executa	1..*	Payment
Staff	1..1	Realiza	0..*	Task

Project	<i>projectID, name, description, startDate, endDate, status, budget</i>
Company	<i>companyID, name, nipc, address, contact, email, sector, password, plan</i>
Staff	<i>staffID, name, profession, nif, email, contact</i>
Offer	<i>offerID, date, status</i>
Task	<i>taskID, title, description, startDate, endDate, status</i>
Chat	<i>chatID</i>
Message	<i>messsageID, text</i>
File	<i>fileID, name, file</i>
Review	<i>reviewID, stars, description, date</i>
Payment	<i>paymentID, plan, value</i>

Tabela 8 - Atributos das Entidades

8.1.4 Documentação de Atributos

Cada atributo apresentado de seguida contém o seu respetivo tipo de dados, tamanho e o tipo de valores que permite receber. Na tabela em baixo serão assim descritos com as respetivas propriedades, de modo a serem devidamente validados e armazenados.

Entidade	Atributos	Descrição	Tipo de Dados e Tamanho		Null - Allowed	Multi - Valued
Project	<i>projectID</i>	Identificador único do projeto	6 caracteres variáveis		Não	Não
	<i>name</i>	Identifica o nome do projeto	64 carateres variáveis		Não	Não
	<i>description</i>	Descrição do projeto	256 caracteres variáveis		Não	Não
	<i>status</i>	Estado do projeto	Enumeração		Não	Não
	<i>budget</i>	Orçamento do projeto	19 números inteiros		Sim	Não
	<i>startDate</i>	Data de início do projeto	Tipo data		Não	Não
	<i>endDate</i>	Data final do projeto	Tipo data		Sim	Não
Company	<i>companyID</i>	Identificador único da empresa	6 caracteres variáveis		Não	Não
	<i>name</i>	Nome da empresa	64 carateres variáveis	Não	Não	

	<i>taxId</i>	Identificação fiscal da empresa	9 números inteiros	Não	Não	
	<i>address</i>	Endereço da empresa	64 carateres variáveis	Não	Não	
	<i>contact</i>	Contacto da empresa	9 números inteiros	Não	Sim	
	<i>email</i>	Email da empresa	64 carateres variáveis	Não	Não	
	<i>sector</i>	Área da empresa	64 carateres variáveis	Não	Não	
	<i>plan</i>	Plano de assinatura da empresa	Enumeração	Não	Não	
	<i>password</i>	Password da empresa	64 carateres variáveis	Não	Não	
Staff	<i>staffID</i>	Indentificador único do funcionário	6 caracteres variáveis	Não	Não	
	<i>name</i>	Nome do funcionário	64 carateres variáveis	Não	Não	
	<i>profession</i>	Profissão do funcionário	64 carateres variáveis	Não	Não	
	<i>nif</i>	Número de identificação fiscal do funcionário	9 números inteiros	Não	Não	
	<i>email</i>	Email do funcionário	64 carateres variáveis	Não	Não	
	<i>contact</i>	Número de telemóvel do funcionário	9 números inteiros	Não	Sim	
	<i>password</i>	Password do funcionário	64 carateres variáveis	Não	Não	
Offer	<i>offerID</i>	Indentificador único da oferta	6 caracteres variáveis	Não	Não	
	<i>date</i>	Data da oferta	Tipo data		Não	Não
	<i>status</i>	Estado da oferta	Enumeração		Não	Não
	<i>description</i>	Descrição da oferta	256 caracteres variáveis		Não	Não
Task	<i>taskID</i>	Identificador único da Tarefa	6 caracteres variáveis	Não	Não	
	<i>title</i>	Titulo da oferta	64 carateres variáveis		Não	Não
	<i>description</i>	Descrição da tarefa	256 caracteres variáveis		Não	Não
	<i>status</i>	Estado da tarefa	Enumeração		Não	Não

	<i>date</i>	Data da tarefa	Tipo data		Não	Não
Review	<i>reviewID</i>	Identificador único da Review	6 caracteres variáveis	Não	Não	
	<i>description</i>	Descrição da Review	256 caracteres variáveis		Não	Não
	<i>stars</i>	Estrelas da revisão	Enumeração		Não	Não
	<i>date</i>	Data em que uma avaliação foi redigida	Tipo data		Não	Não
File	<i>fileID</i>	Identificador único de Ficheiro	6 caracteres variáveis		Não	Não
	<i>name</i>	Nome do ficheiro	64 caracteres variáveis		Não	Não
	<i>file</i>	Ficheiro a ser carregado	Máx.16777216 bytes tipo binário		Não	Não
Message	<i>messageID</i>	Identificador único de Mensagem	6 caracteres variáveis		Não	Não
	<i>text</i>	Texto escrito por utilizador numa mensagem	256 caracteres variáveis		Não	Não
Chat	<i>chatID</i>	Identificador único de Chat	6 caracteres variáveis		Não	Não
Payment	<i>paymentID</i>		6 caracteres variáveis		Não	Não
	<i>value</i>		12 caracteres tipo Float		Não	Não
	<i>data</i>		Tipo data		Não	Não
	<i>plan</i>		Enumeração		Não	Não

Tabela 9 - Documentação de Atributos

8.1.5 Atribuição das chaves primárias-

Nesta fase do desenvolvimento do desenho conceptual, já conseguimos determinar as chaves primárias de cada entidade e, desta forma, a forma mais intuitiva para as demonstrar será num diagrama ER.

Nome da Entidade	Chave Primária
Projeto	<i>projetoID</i>
Empresa	<i>empresaID</i>
Funcionário	<i>funcionarioID</i>
Proposta	<i>propostaID</i>

Tarefa	<i>taskID</i>
Message	<i>messageID</i>
Chat	<i>chatID</i>
Review	<i>reviewID</i>
File	<i>fileID</i>
Payment	<i>paymentID</i>

Tabela 10 – Atribuição de chaves primárias

Na figura abaixo encontramos um diagrama ER com relacionamentos e chaves primárias adicionadas.

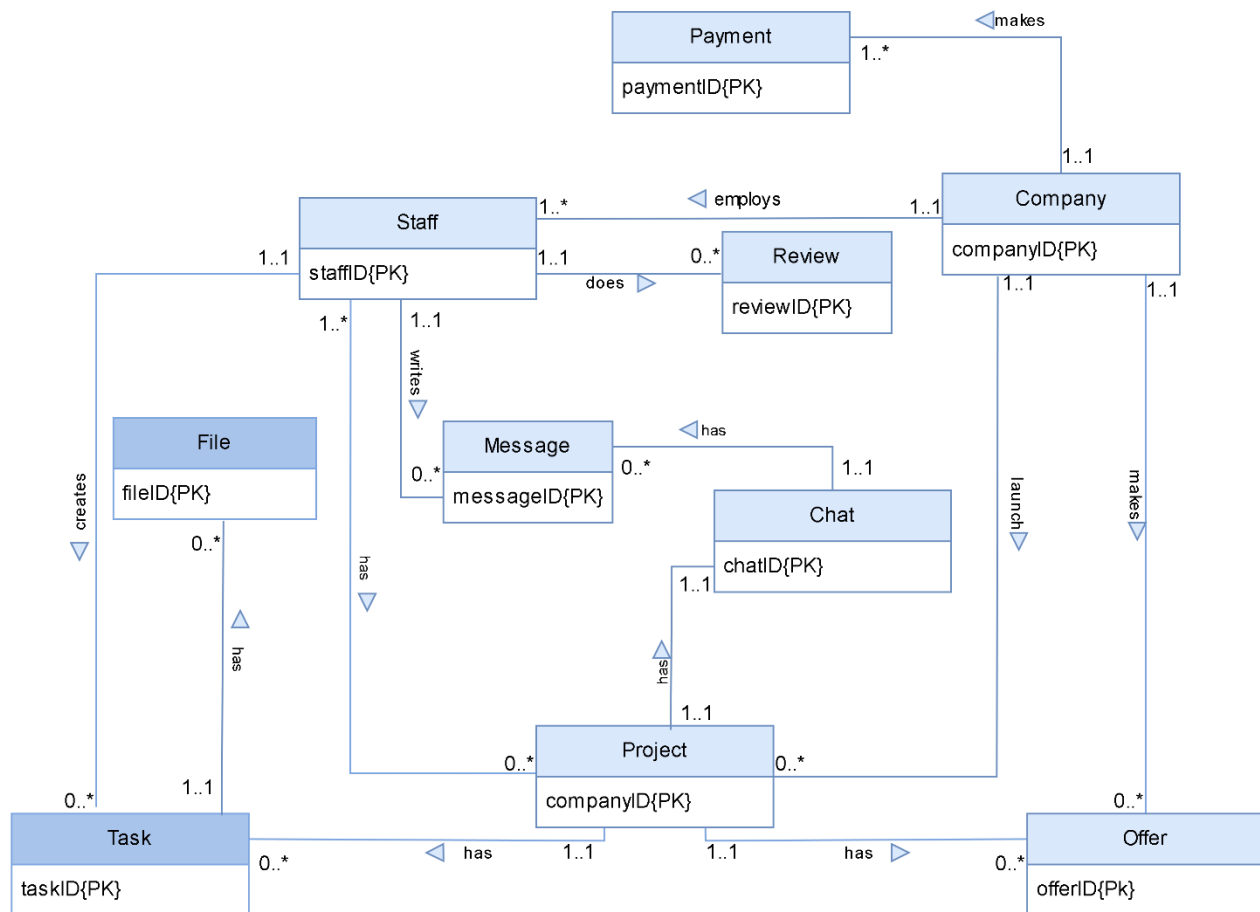


Figura 10- Diagrama ER com chaves primárias

8.1.6 Verificar o modelo para redundância

Nesta etapa, é verificada a ocorrência de redundância e removida quando se confirma a sua existência. As três atividades desta etapa são as seguintes:

1. Reexaminar relações um-para-um (1:1)

Mediante a abordagem seguida, não existem relações um-para-um pelo que será possível verificar a inexistência de redundância ou, por outro lado, o sucesso sobre esta atividade.

2. Remover relacionamentos redundantes

Após uma intensa e longa análise, confirmou-se que não existe redundância em quaisquer relacionamentos deste caso de estudo. Todos os relacionamentos são estritamente essenciais e requisito obrigatório para a conclusão deste projeto.

3. Considerar a dimensão do tempo

Após a análise, verificamos que as relações descritas não possuem redundâncias quando consideradas sob a dimensão do tempo.

8.1.7 Apresentação do Desenho Conceptual

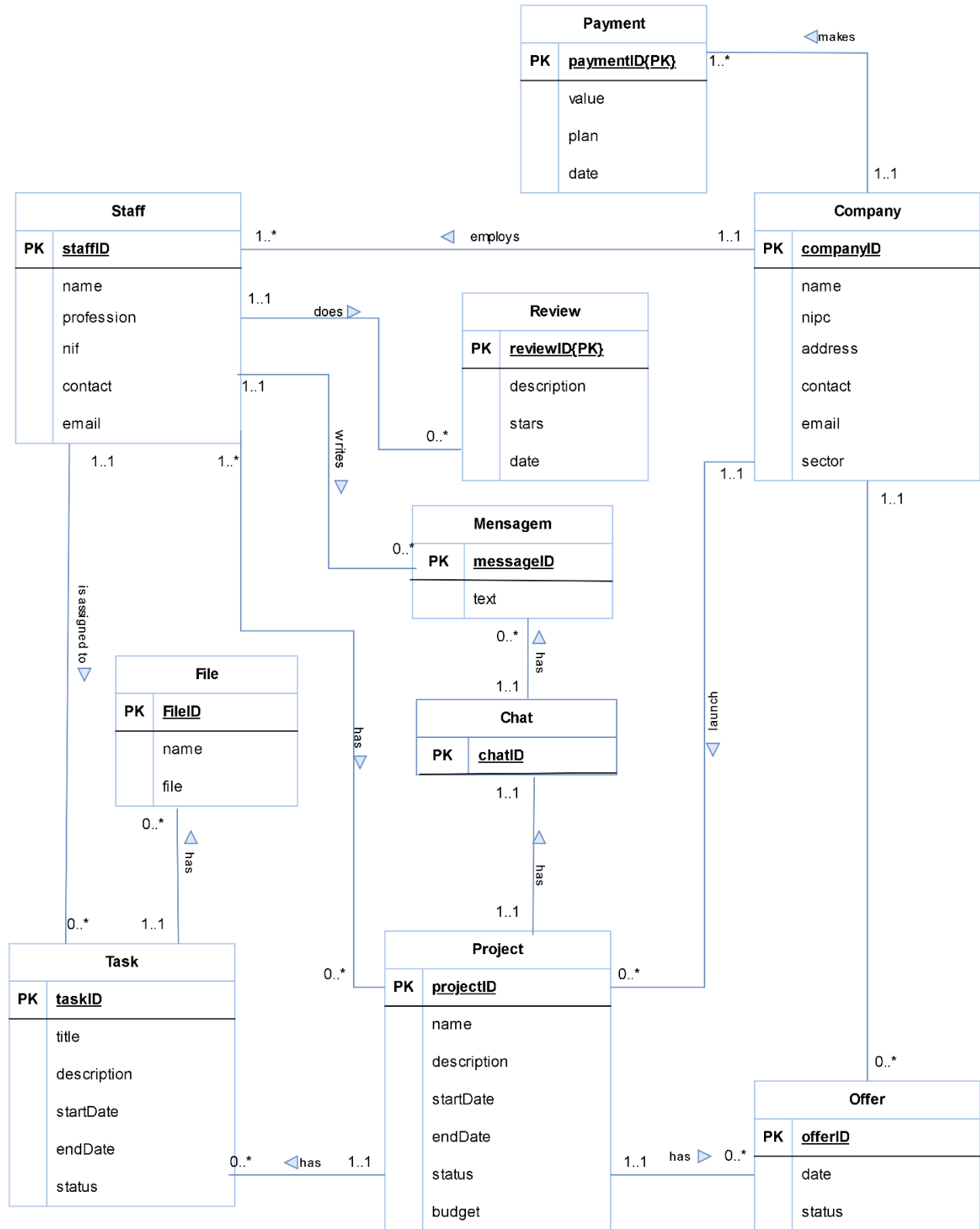


Figura 11 – Diagrama Conceptual

8.2 Modelo Lógico

1. Derivar relações para o modelo de dados lógico
2. Validar relações usando normalização
3. Verificar restrições de integridade

8.2.1 Derivação das Relações

8.2.1.1 Entidades Fortes

É considerada forte aquela entidade que pode ser identificada exclusivamente pelos seus próprios atributos, sem depender de outra entidade. Neste contexto, *Company* é a entidade forte do nosso sistema, pois não depende de qualquer uma das outras para existir.

Por outro lado, é considerada uma entidade fraca aquela que não possui uma Primary Key própria e depende de uma entidade forte para ser identificada de forma única.

8.2.1.2 Relacionamentos de um para um (1:1)

Project: Chat

No seguinte relacionamento, cada projeto possui um único chat.

Project {*projectID*, *endDate*, *status*, *name*, *description*, *budget*, *startDate*}

Primary Key *projectID*

Foreign Key *companyID*

Chat {*chatID*}

Primary Key *chatID*

Foreign Key *projectID*

8.2.1.3 Relacionamentos de um para muitos (1:*)

Staff: Review *

Neste relacionamento, um staff existe em zero ou mais Reviews, no entanto uma Review contém apenas um staff.

Review {*reviewID*, *stars*, *date*, *description*}

Primary Key *reviewID*

Foreign Key *reviewerID*, *revieweeID*

Staff {*staffID*, *profession*, *nif*, *contact*, *email*, *name*, *password*}

Primary Key *staffID*

Foreign Key *companyID*

Staff: Message *

No relacionamento abaixo, um registo de Mensagem é único e é executado por um único staff, no entanto, um staff pode registar múltiplas mensagens.

Staff {*staffID*, *profession*, *nif*, *contact*, *email*, *name*, *password*}

Primary Key *staffID*

Foreign Key *companyID*

Message {*messageID*, *text*}

Primary Key *messageID*

Foreign Key *chatID*, *staffID*

Staff: Task *

No relacionamento abaixo, uma Task é única e existe num determinado staff, no entanto, um staff pode conter múltiplas Tasks.

Staff {*staffID*, *profession*, *nif*, *contact*, *email*, *name*, *password*}

Primary Key *staffID*

Foreign Key *companyID*

Task {*taskID*, *startDate*, *endDate*, *status*, *title*, *description*}

Primary Key *taskID*

Foreign Key *projectID*, *staffID*

Task: File *

No seguinte relacionamento, uma Task existe em zero ou mais files, no entanto, um file contém apenas uma Task.

Task {*taskID*, *startDate*, *endDate*, *status*, *title*, *description*}

Primary Key *taskID*

Foreign Key *projectID*, *staffID*

File {*fileID*, *name*, *file*}

Primary Key *fileID*

Foreign Key *taskID*

Project: Task *

Relação onde um project existe em zero ou mais Tasks, no entanto, uma Task contém apenas um project.

Project {*projectID, endDate, status, name, description, budget, startDate*}

Primary Key *projectID*

Foreign Key *companyID*

Task {*taskID, startDate, endDate, status, title, description*}

Primary Key *taskID*

Foreign Key *projectID, staffID*

Project: Offer *

No subsequente relacionamento, um registo de Offer é único e é executado por um único project, no entanto, um project pode registar múltiplas Offers.

Project {*projectID, endDate, status, name, description, budget, startDate*}

Primary Key *projectID*

Foreign Key *companyID*

Offer {*offerID, date, status, description*}

Primary Key *offerID*

Foreign Key *companyID, projectID*

Company: Offer *

No relacionamento que se persegue, uma Company existe em zero ou mais Offers, no entanto, uma Offer contém apenas uma Company

Company {*companyID, name, taxID, address, contact, email, sector, plan, password*}

Primary Key *companyID*

Foreign Key

Offer {*offerID, date, status, description*}

Primary Key *offerID*

Foreign Key *companyID, projectID*

Company: Project *

Neste relacionamento, uma Company existe em zero ou mais Projects, no entanto um project contém apenas uma Company.

Company {*companyID, name, taxID, address, contact, email, sector, plan, password*}

Primary Key *companyID*

Foreign Key

Project {*projectID, endDate, status, name, description, budget, startDate*}

Primary Key *projectID*

Foreign Key *companyID*

Company: Staff *

Relação onde uma Company existe em zero ou mais staffs, no entanto, um staff contém apenas uma Company.

Company {*companyID*, *name*, *taxID*, *address*, *contact*, *email*, *sector*, *plan*, *password*}

Primary Key *companyID*

Foreign Key

Staff {*staffID*, *profession*, *nif*, *contact*, *email*, *name*, *password*}

Primary Key *staffID*

Foreign Key *companyID*

Chat: Message *

Na contemporânea relação, um chat existe em zero ou mais Messages, no entanto, uma Message contém apenas um chat.

Chat {*chatID*}

Primary Key *chatID*

Foreign Key *projectID*

Message {*messageID*, *text*}

Primary Key *messageID*

Foreign Key *chatID*, *staffID*

Company: Payment *

Neste relacionamento, uma Company existe em zero ou mais Payment, no entanto um Payment contém apenas uma Company.

Company {*companyID, name, taxID, address, contact, email, sector, plan, password*}

Primary Key *companyID*

Foreign Key

Payment {*paymentID, plan, date, value*}

Primary Key *paymentID*

Foreign Key *companyID*

8.2.1.4 Relacionamentos de muitos para muitos (*:*)

Staff *: Project *

No imediato relacionamento, zero ou mais staffs existe em zero ou mais Projects e vice-versa.

Staff {*staffID, profession, nif, contact, email, name, password*}

Primary Key *staffID*

Foreign Key *companyID*

Project {*projectID, endDate, status, name, description, budget, startDate*}

Primary Key *projectID*

Foreign Key *companyID*

8.2.2 Normalização

O processo de normalização surge como uma atividade associada ao Desenho Lógico que visa validar se as relações obtidas anteriormente são válidas. Este processo de validação assenta no estudo das dependências funcionais e transitivas existentes entre atributos de uma relação.

A normalização de uma tabela na 1.ª Forma Normal (1FN) exige que a tabela tenha uma estrutura bidimensional correta, ou seja, cada linha deve corresponder a um só registo e cada coluna a um só campo.

O objetivo é eliminar redundância e introduzir dados apropriados nas colunas vazias das linhas que contêm grupos repetidos.

No processo de normalização, ao analisar a relação entre Staff e Project reparou-se a ligação entre as duas entidades não pode existir diretamente, o que ocorre porque um funcionário pode estar associado a vários projetos e um projeto pode ter vários funcionários. Esta relação foi solucionada com a introdução da tabela StaffOnProject que contém as Primary Keys de ambas as entidades, permitindo que a ligação seja feita de uma forma correta e normalizada.

A 2.^a forma normal diz que a tabela tem de estar na 1FN e que cada atributo não chave tem de ser funcionalmente dependente da totalidade da Primary Key e não apenas de uma parte dessa chave.

Assim depois de identificada a Primary Key de uma tabela, pode dar-se um dos dois casos: A Primary Key é constituída por um só atributo (chave elementar). Neste caso, a tabela está seguramente na 2FN (nenhum atributo depende de uma parte da chave, visto que a chave não é composta por partes);

A Primary Key é constituída por mais que um atributo (Primary Key composta). Neste caso, se existe algum ou alguns atributos que dependem de uma parte da chave (ou seja, de algum atributo que constitui a chave), então a tabela não está na 2FN.

A 3.^a Forma Normal (3FN) diz que a tabela tem de estar na 2FN e que nenhum atributo não chave pode depender funcionalmente de algum outro atributo que não seja a Primary Key.

Portanto, para normalizar uma tabela de acordo com a 3FN, deve-se analisar todos os atributos não chave e verificar se existem dependências transitivas sobre a Primary Key, removê-las e colocá-las numa nova relação.

8.2.3 Apresentação do desenho lógico

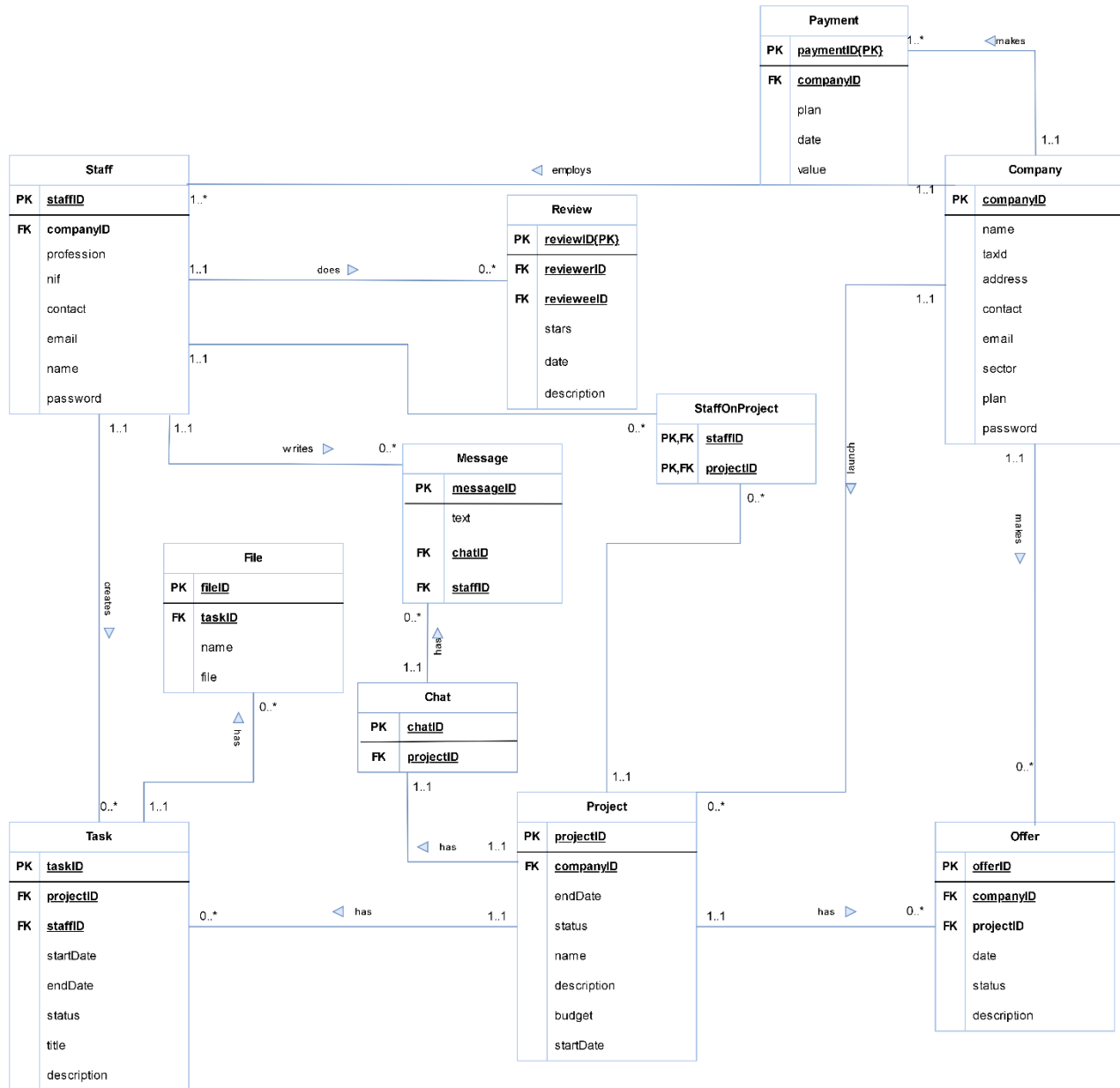


Figura 12 – Desenho lógico

8.3 Desenho Físico

Abaixo temos um diagrama criado pelo SQL Management Studio.

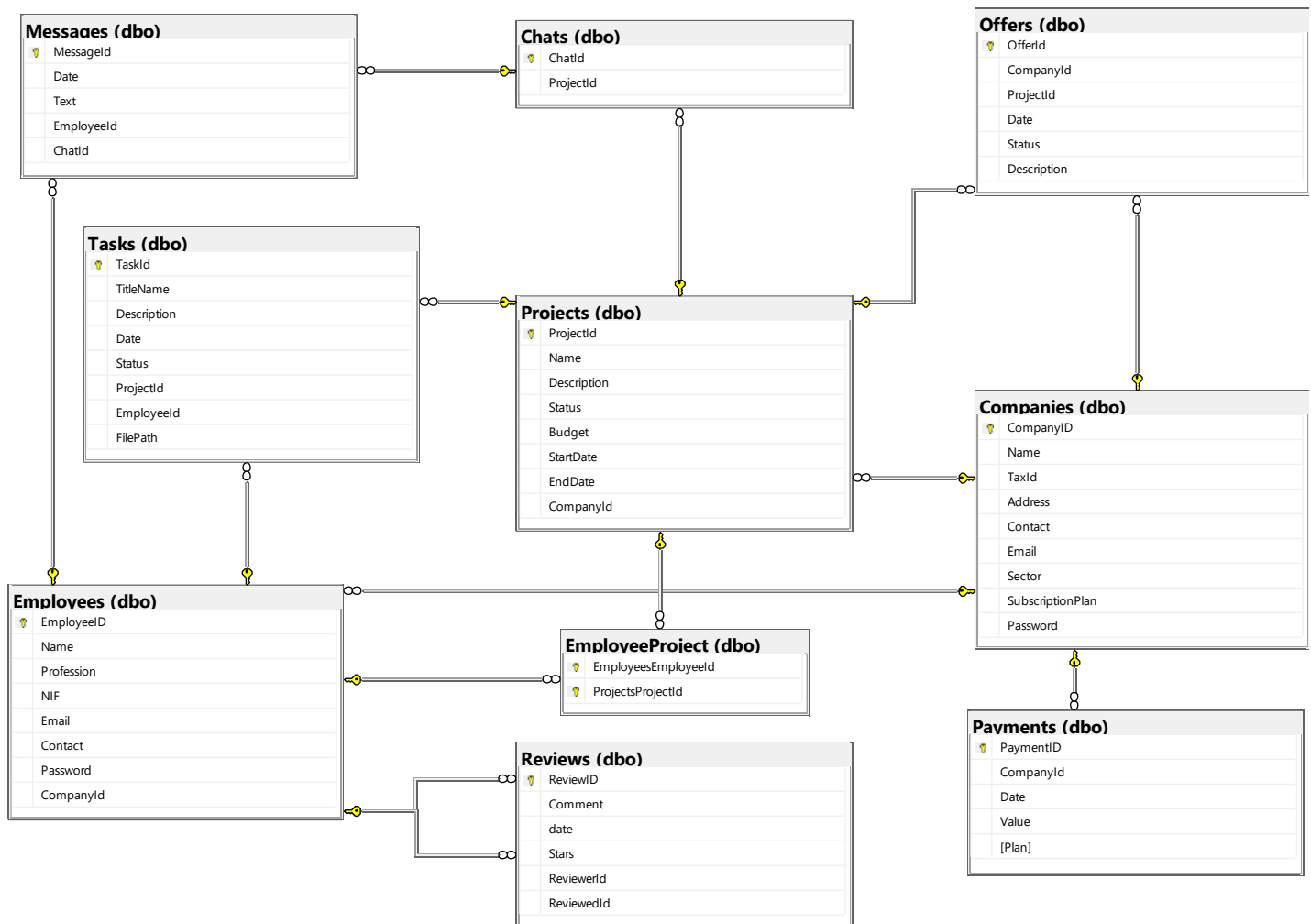


Figura 13 – Desenho Físico da BD

9. Mockups

Os Mockups apresentados neste projeto tem como objetivo fornecer uma visão clara e detalhada da interface do sistema, tanto para a versão web como para a mobile. Estes servem como representação visual das funcionalidades e da experiência do utilizador.

A versão web inclui o acesso a todas as funcionalidades do sistema, permitindo assim a cada empresa gerir todos os seus funcionários e todos os seus projetos, como também a gestão de assinaturas e informações da mesma.

A versão mobile permite representar a utilização do funcionário que terá acesso aos seus projetos, bem como a todas as funcionalidades de comunicação e envio de documentação dentro dos mesmos.

9.1.1 Página Inicial

A figura representa a página inicial da aplicação onde irá conter uma breve introdução do produto que a aplicação promove bem como os seus benefícios. Apresenta todas as ligações necessárias para navegar dentro do sistema.

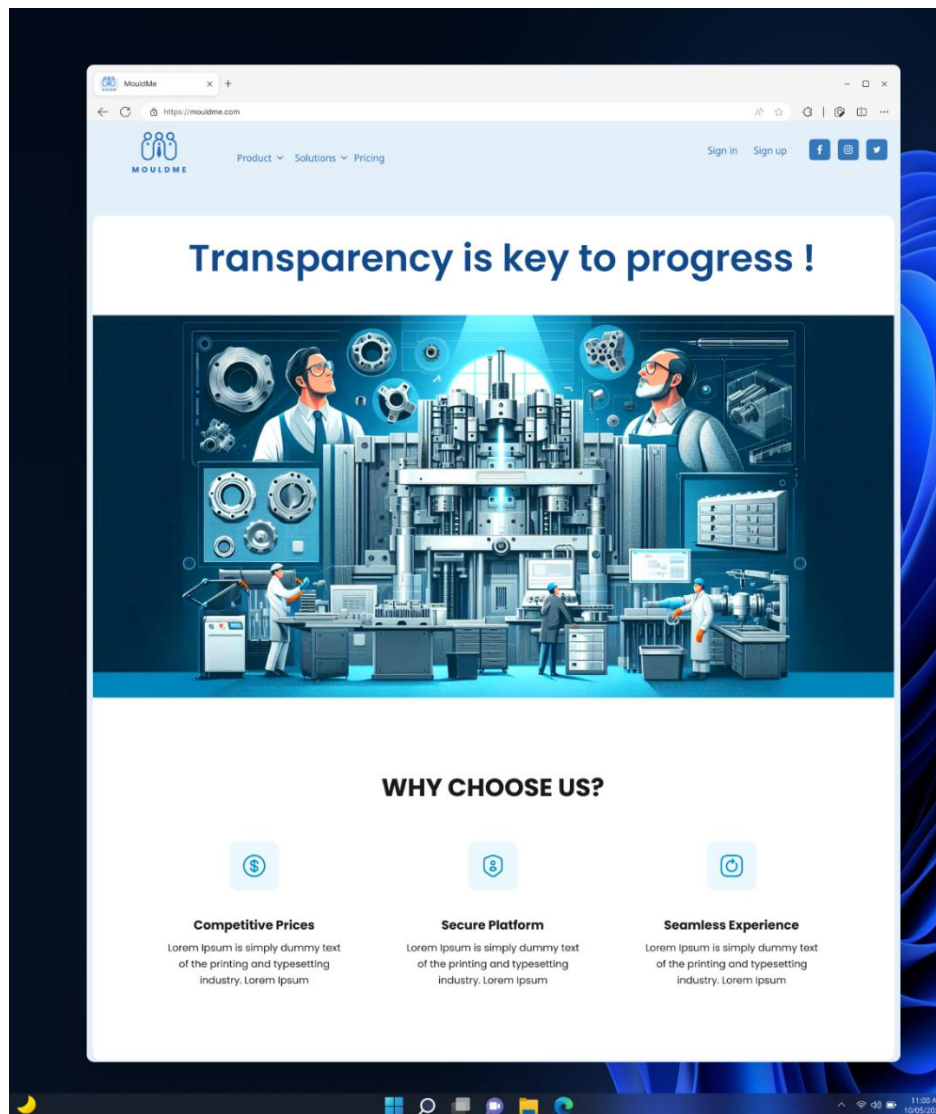


Figura 14 – HomePage

9.1.2 Login

As figuras representam os ecrãs de login da aplicação web e mobile respetivamente, que possibilitam a recuperação da password e o acesso á aplicação, o mesmo acontece na aplicação movel que é a apresentação do login do funcionário, com a possibilidade recuperar password ou aceder ao sistema.

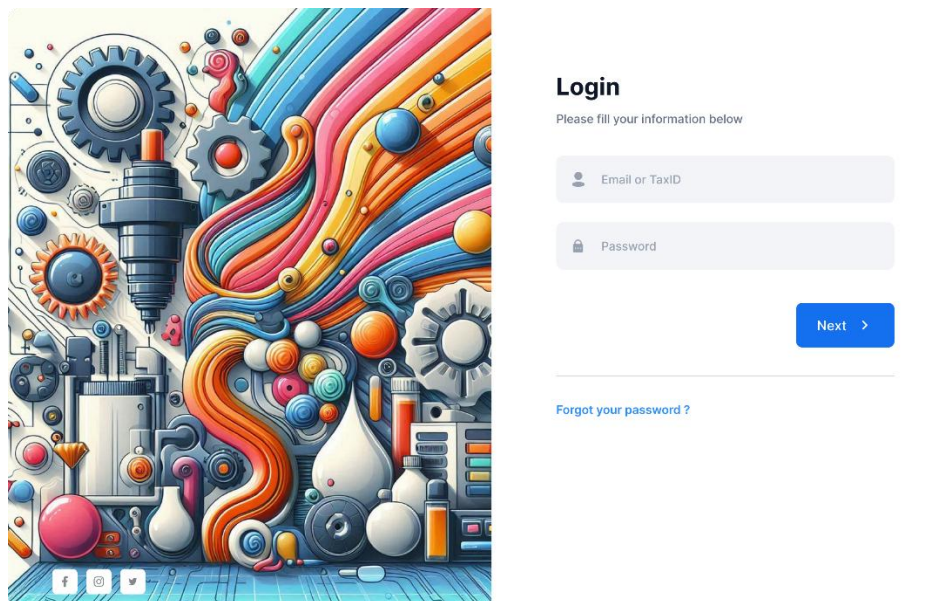


Figura 15 – Login – Web

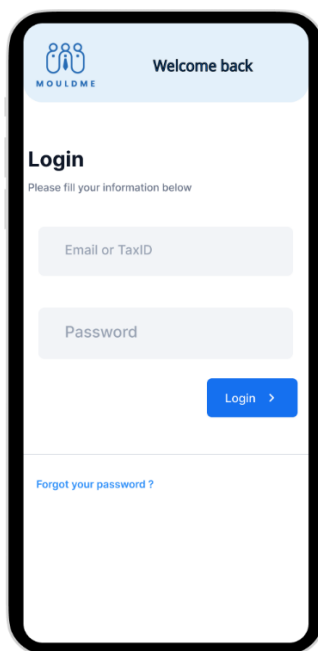


Figura 11 – Login – Mobile

9.1.3 Recuperar Password

Nas seguintes figuras podemos observar os ecrãs de recuperação de password web e mobile, onde acontece a alteração por parte do utilizador após a necessidade de alterar a mesma.



Recover your Password

Please fill your information below

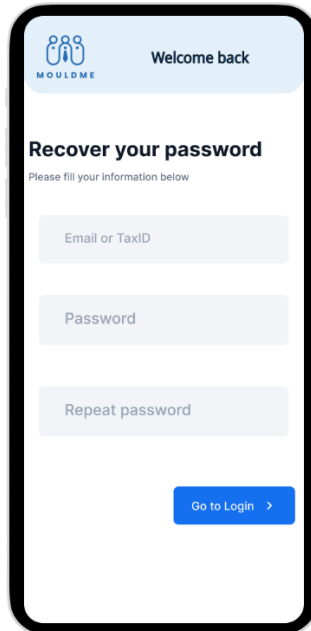
Email or TaxID

Password

Repeat Password

Finish >

Figura 16 - Recuperar Password - Web



MOULME Welcome back

Recover your password

Please fill your information below

Email or TaxID

Password

Repeat password

Go to Login >

Figura 17 - Recuperar Password - Mobile

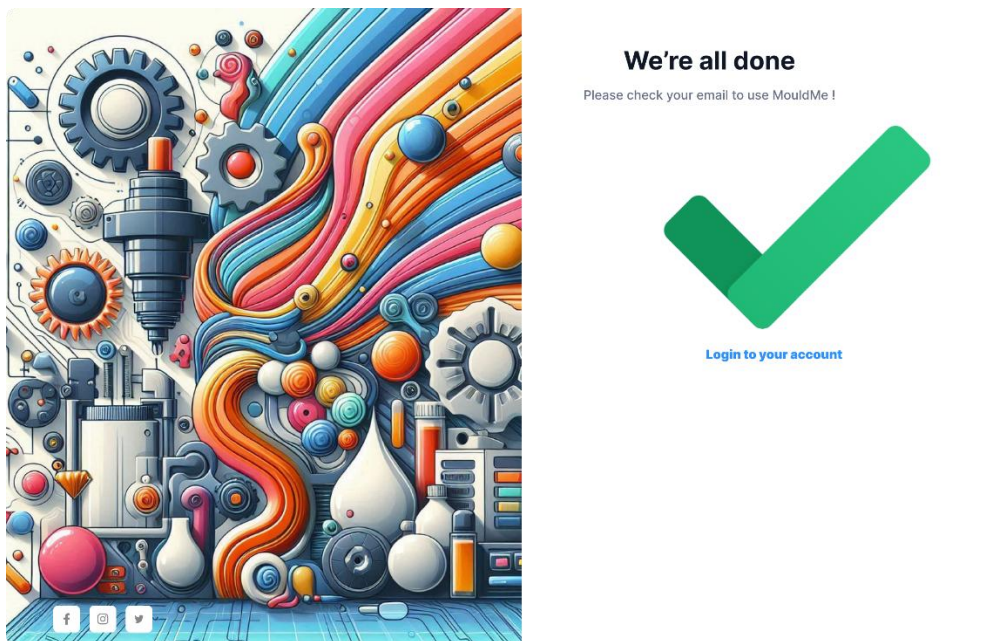


Figura 18 – Tudo pronto (Login)

9.1.4 Registrar Entidade

Nas figuras 14 e 17 estão os passos que a empresa consegue visualizar ao efetuar o registo na aplicação, pois apresenta todos os campos necessários para o preenchimento. De seguida as figuras 15 e 16 apresentam os passos seguintes que são a escolha do plano de assinatura e o pagamento do mesmo.

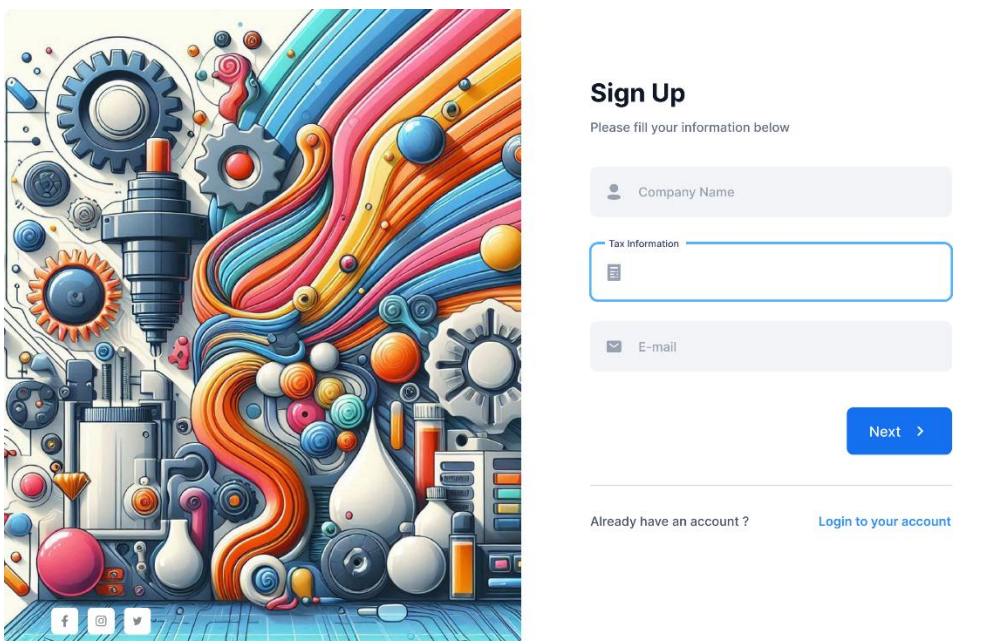


Figura 19-Registo

PRICING

Simple, transparent pricing

Lorem ipsum dolor sit amet consectetur adipiscing elit dolor posuere vel venenatis eu sit massa volutpat.

For individuals

Basic

Lorem ipsum dolor sit amet

\$99

/monthly

What's included

All analytics features

Up to 250,000 tracked visits

Normal support

Up to 3 team members

Get started

For startups

Pro

Popular

Lorem ipsum dolor sit amet

\$199

/monthly

What's included

All analytics features

Up to 1,000,000 tracked visits

Premium support

Up to 10 team members

Get started

For big companies

Enterprise

Lorem ipsum dolor sit amet

\$399

/monthly

What's included

All analytics features

Up to 5,000,000 tracked visits

Dedicated support

Up to 50 team members

Get started

Figura 20 - Escolher plano

Express checkout

shop Pay

Google Pay

PayPal

OR

Payment

All transactions are secure and encrypted

Credit or debit card

Card number

Expiration date (MM/YY)

Security code

Name on card

VISA

MasterCard

Amex

PayPal

PayPal

Pay now

Professional Plan

\$199.00

Subtotal

\$199.00

Estimated taxes ⓘ

\$5.00

Total

\$204.00

Including \$5.00 in taxes

Figura 21 – Pagamento de Plano



Sign Up

Please fill your information below

^

We do injection Systems

We do Molds

We need a plastic Product

[Finish >](#)

Figura 22 - Registo

9.1.5 Perfil Público

As figuras seguintes apresentam a representação dos perfis de utilizador ou funcionário e o perfil da entidade, que conta com os membros associados ao mesmo e os projetos em participaram. O funcionário contém a também a avaliação que obteve em consequência das avaliações recebidas.

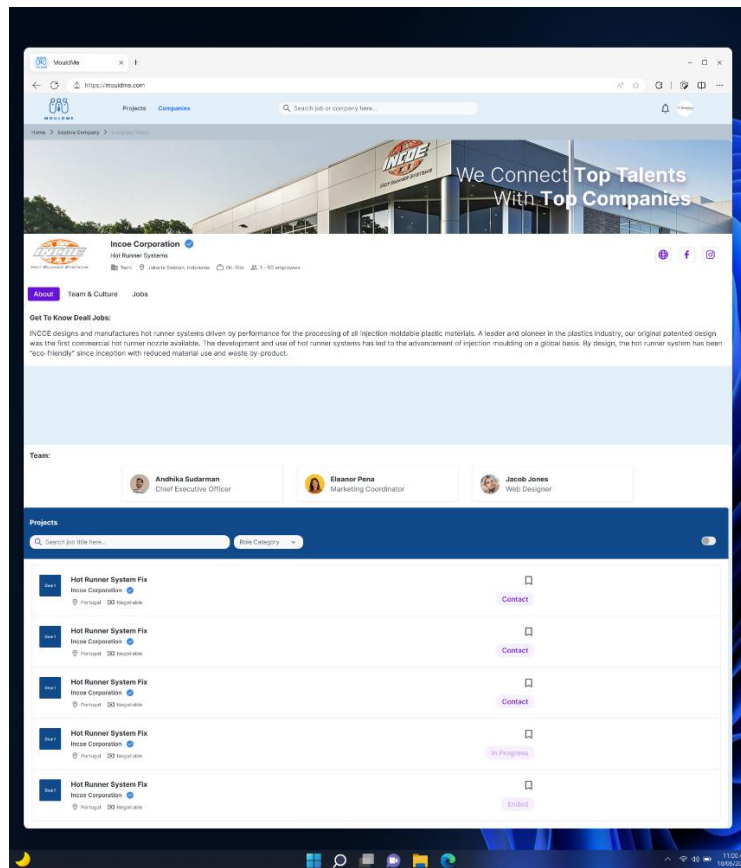


Figura 23 - Perfil público da empresa

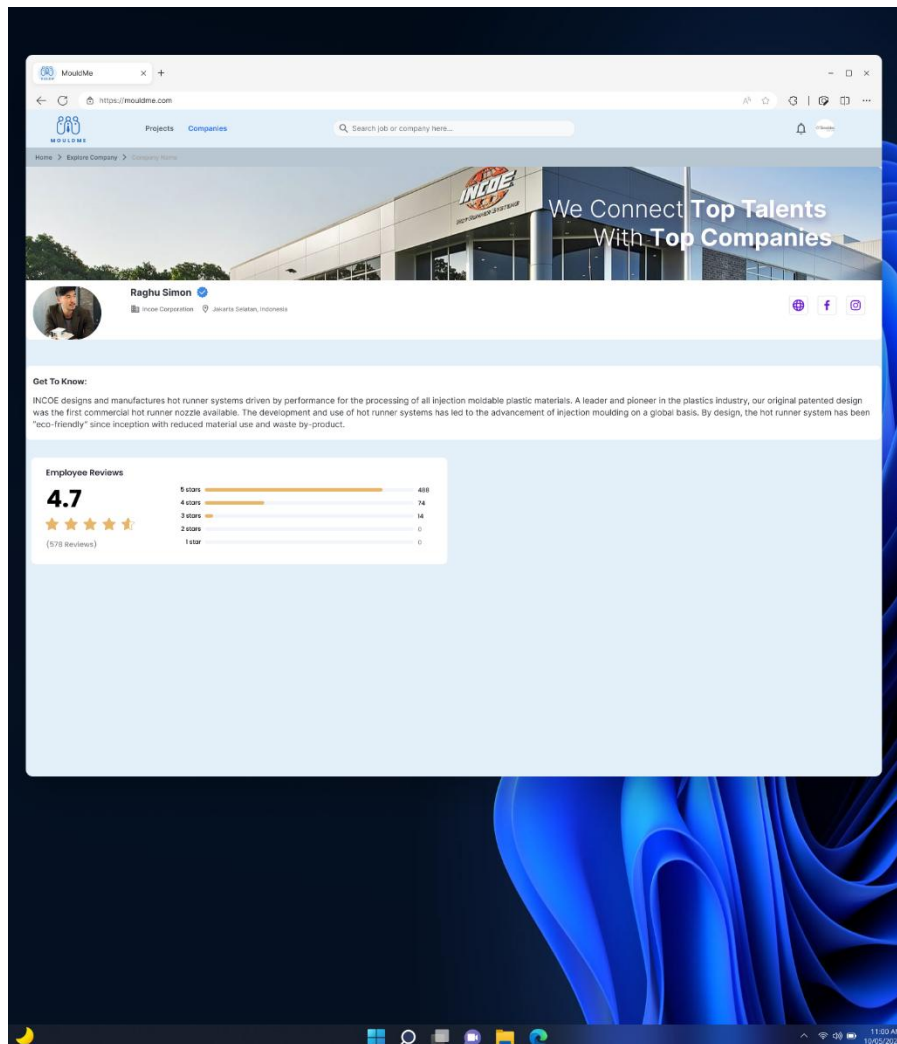


Figura 24 – Perfil Público de Funcionário com avaliações

9.1.6 Editar Perfil

Página de perfil de utilizador, com as respetivas opções de alterar as suas informações pessoais, bem como a sua foto e os dados de acesso.

The screenshot shows the 'Company Profile' page in the MouldMe application. The left sidebar contains navigation links: 'My profile', 'Projects', 'Staff', 'Calendar', 'Settings', and a 'Sign out' button. The main content area is titled 'Company Profile' and features a user profile card for 'Simoldes' (Mould Production) with an 'Upload New Photo' button and a 'Delete' button. Below this, there are form fields for 'Company Name' (with a placeholder 'eg. Alaa'), 'Work Area' (with a placeholder 'eg. Mohamoud'), 'User Name' (with a placeholder 'eg. alaa.mohamoud'), 'Email Address', 'Contact', 'Tax ID', and another 'Contact' field. At the bottom, there are fields for 'Location', 'Time Zone', 'Current Password', 'New Password', and 'Confirm New Password'. 'Cancel' and 'Save Changes' buttons are at the bottom right.

Figura 2126 – Editar perfil da Empresa

The screenshot shows the 'User Profile' page in the MouldMe application. The left sidebar is identical to the previous one. The main content area is titled 'User Profile' and features a user profile card for 'Alaa Mohamed' (Product Design) with an 'Upload New Photo' button and a 'Delete' button. Below this, there are form fields for 'First Name' (placeholder 'eg. Alaa'), 'Last Name' (placeholder 'eg. Mohamoud'), 'User Name' (placeholder 'eg. alaa.mohamoud'), 'Email Address', and 'Phone Number'. There is a 'Companies' section listing 'ProCrew', 'Noon', and 'LamasaTech', and an 'About Myself' text area. At the bottom, there are fields for 'Location', 'Time Zone', 'Current Password', 'New Password', and 'Confirm New Password'. 'Cancel' and 'Save Changes' buttons are at the bottom right.

Figura 2722 – Editar perfil de funcionário

9.1.7 Listas

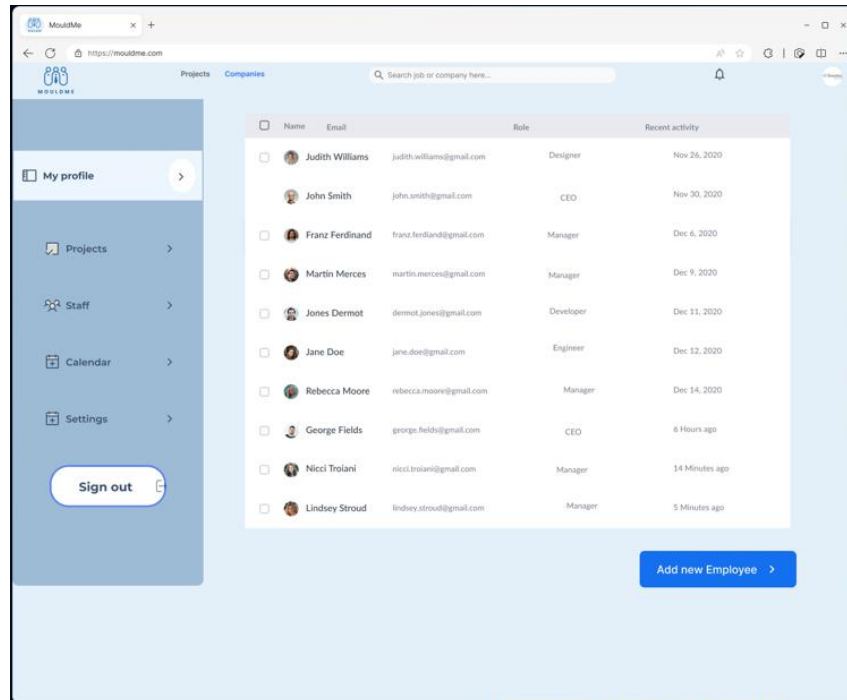


Figura 2328- Lista de funcionários de uma Empresa

Nas seguintes imagens são apresentadas páginas de listagem de todos os projetos atribuídos à empresa, ao funcionário (com a data e o status do mesmo visível) e uma vista mobile do anterior.

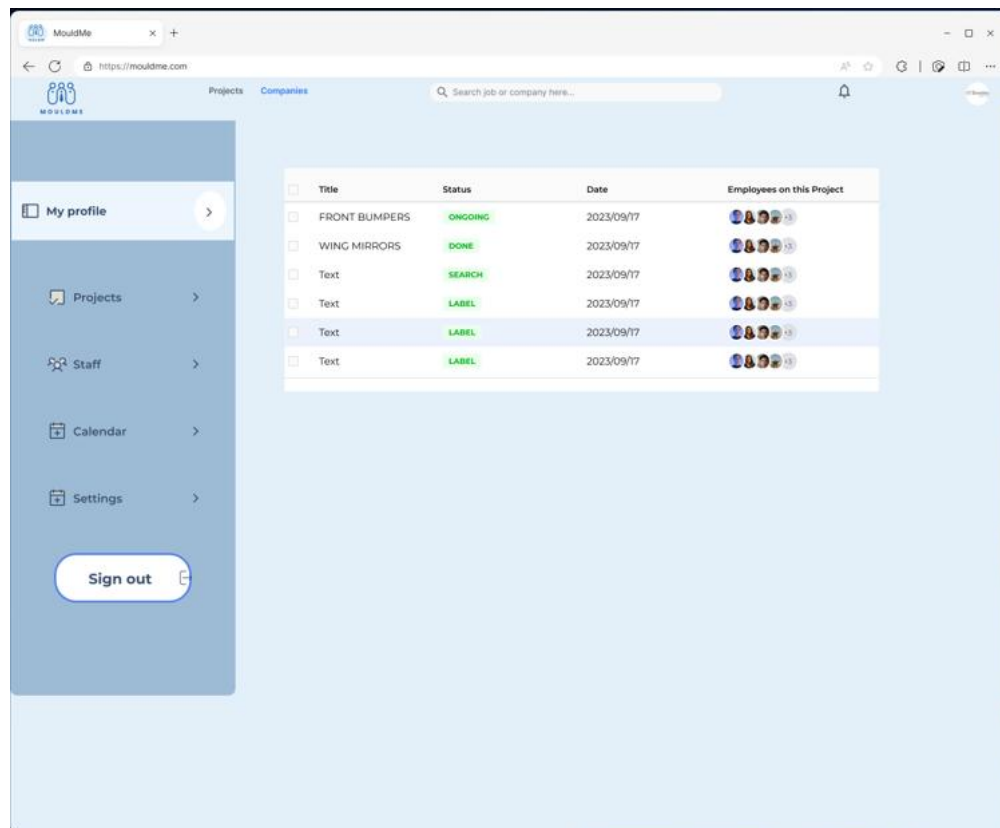


Figura 24 29 – Lista de Projetos de uma Empresa

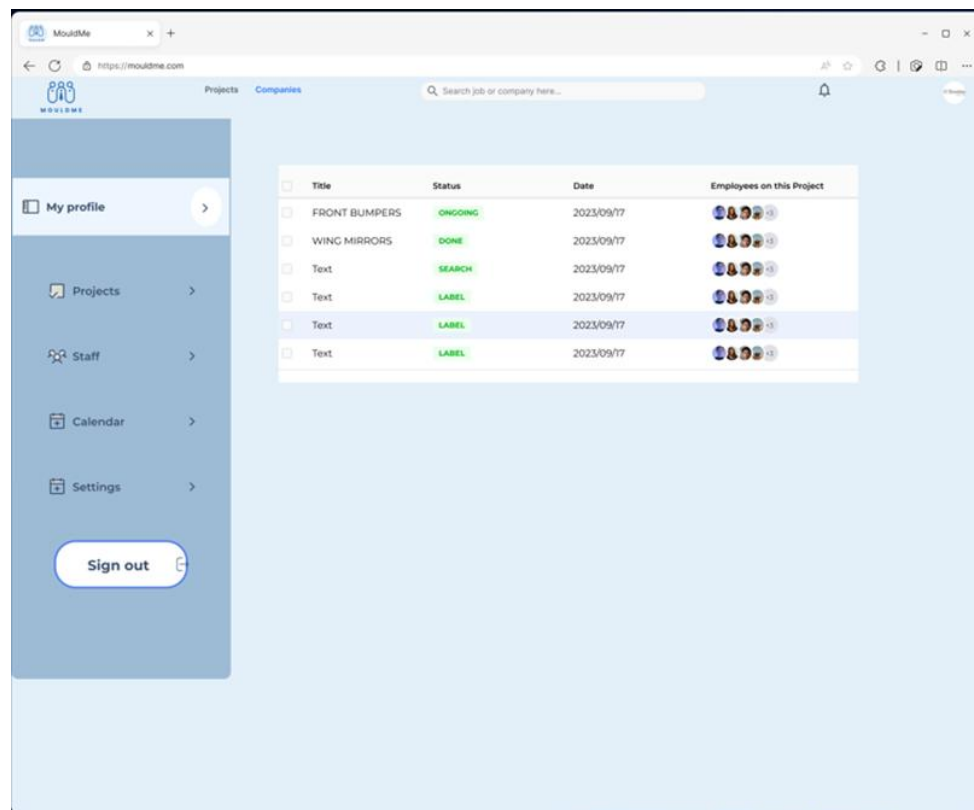


Figura 25 30- Lista de Projetos de um funcionário

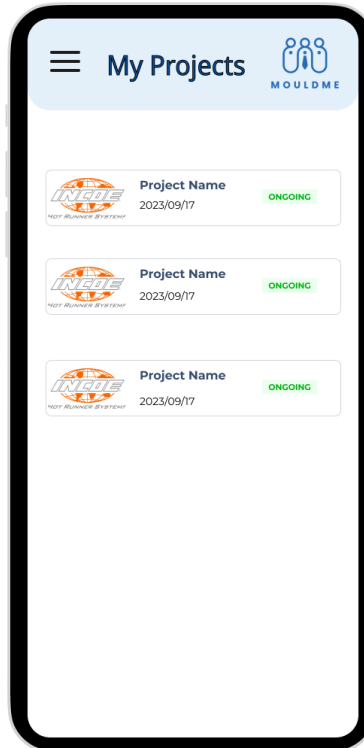


Figura 26 31 - Lista de projetos - Mobile

9.1.8 Projeto

Nas seguintes figuras vemos o ecrã dedicado à criação de projeto e propostas feitas para este.

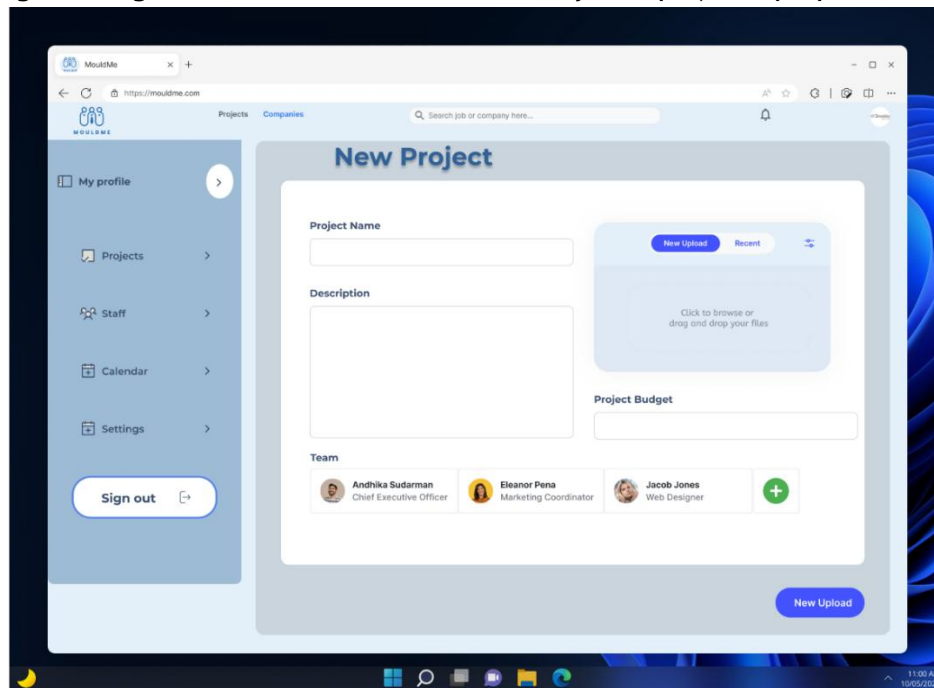
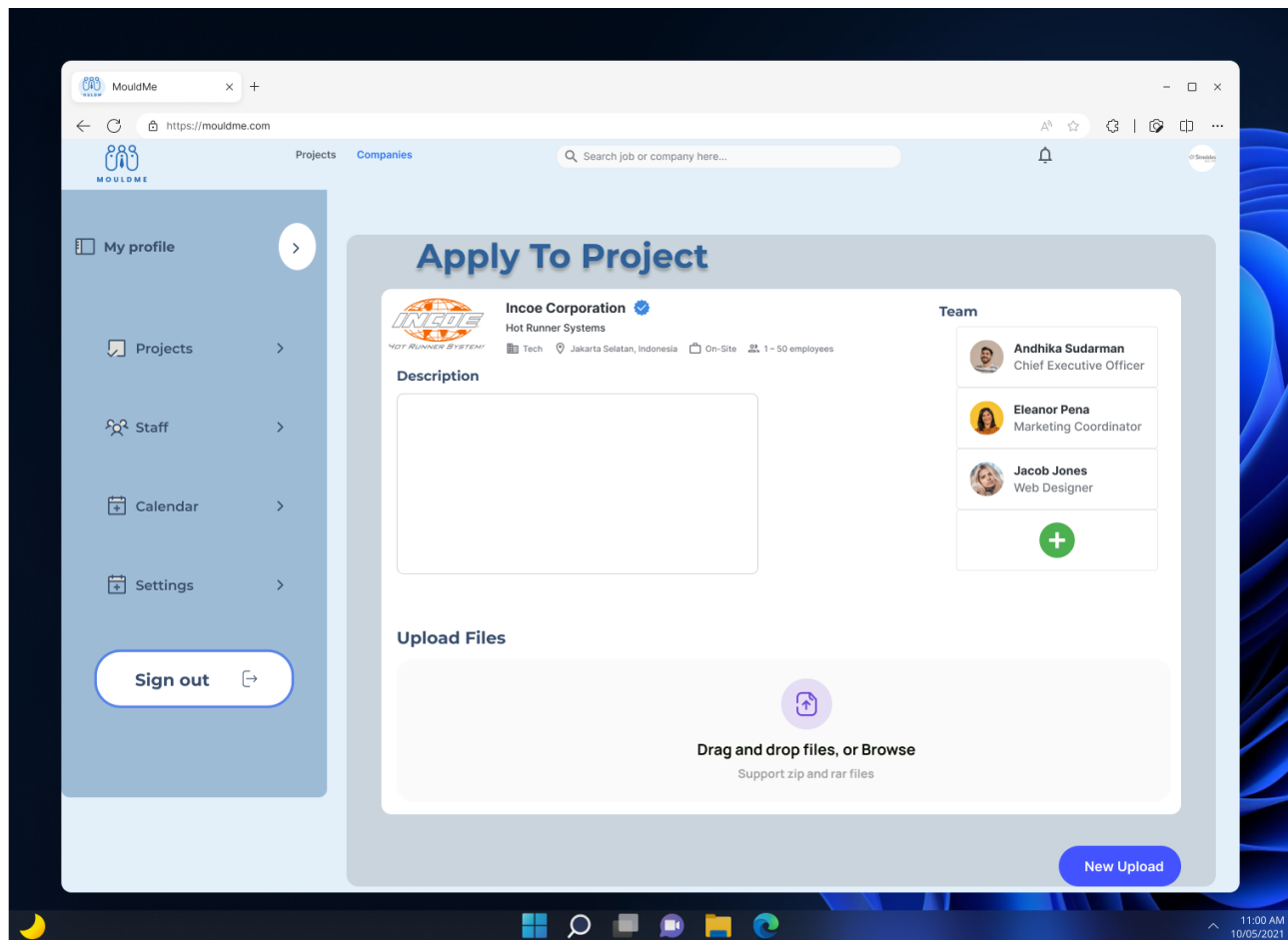


Figura 32 - Novo Projeto

*Figura 33 – Proposta para Projeto*

Nas seguintes figuras verificamos que no projeto, é possível introduzir uma tarefa e verificar as tarefas ativas dentro dele, enviar uma mensagem para o chat, ver antigas tarefas, consultar todos os documentos partilhados neste projeto e a equipa o projeto

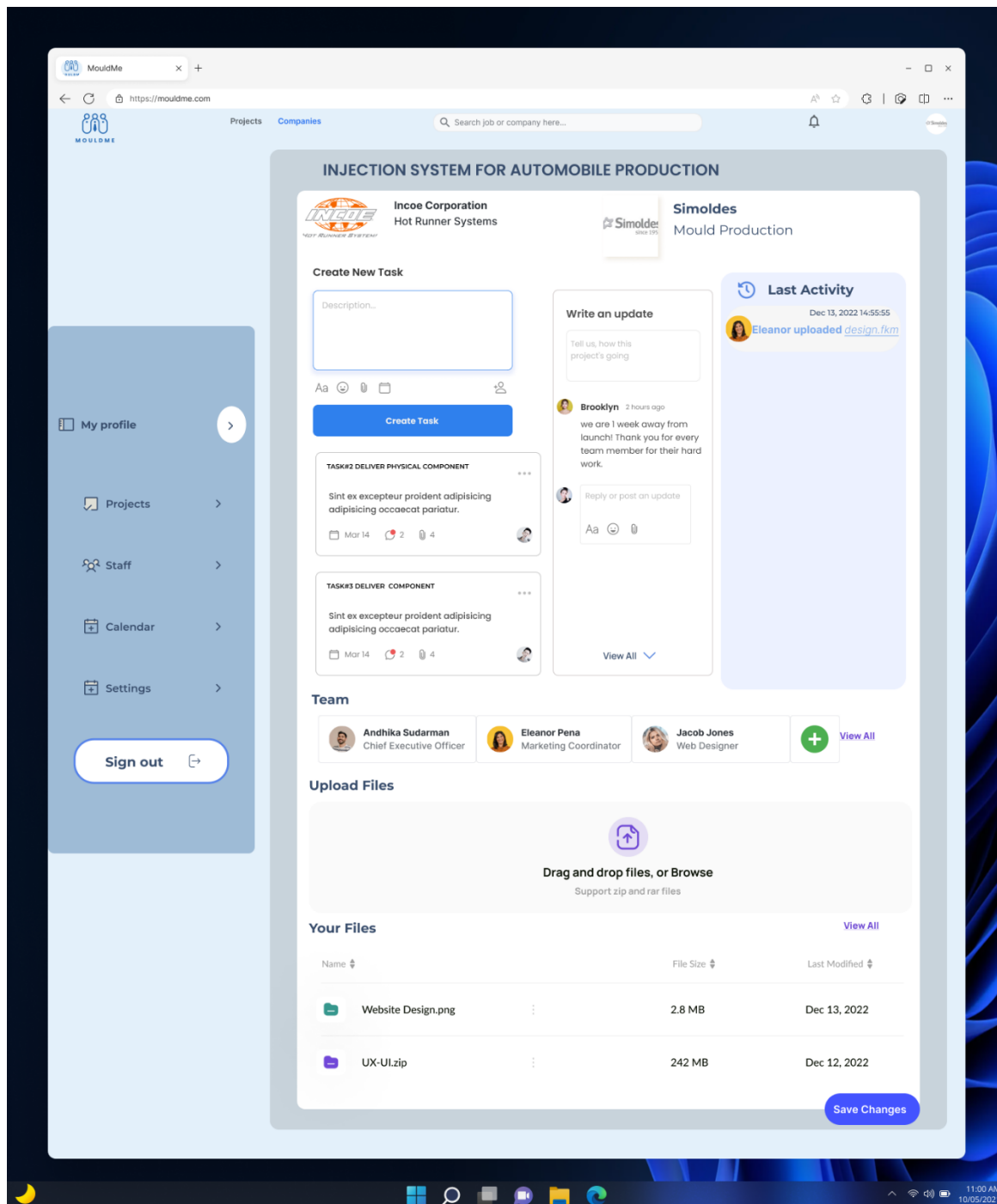


Figura 34 29– Página do Projeto

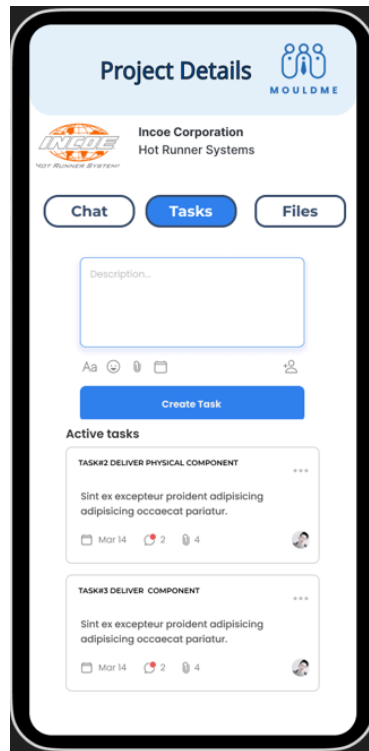


Figura 35 30 – Página do Projeto

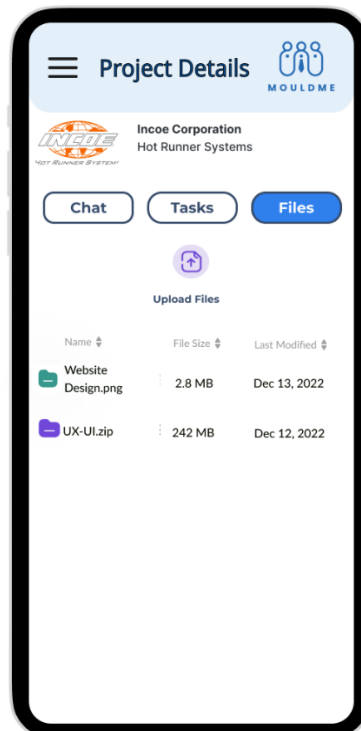


Figura 36 31 – Upload de Documentos em Projeto

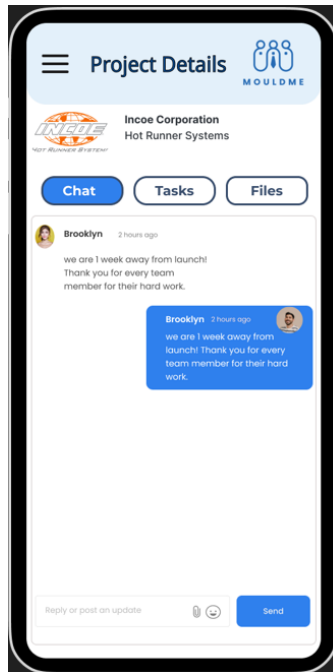


Figura 37 32- Chat de um Projeto - Mobile

9.1.9 Menu

Apresentação do menu lateral da aplicação mobile, que irá permitir navegar dentro dos ecrãs da aplicação.

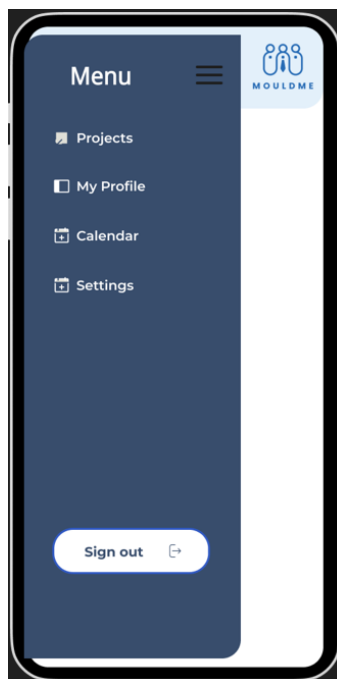


Figura 33 – Aba Lateral

10. Testes

A presente secção tem como objetivo descrever todos os cenários de casos de teste dos respetivos métodos desenvolvidos para a aplicação.

ReviewControllerTest

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC001	Adicionar uma review válida	Base de dados configurada e dados iniciais (empresa e funcionários) definidos	Comment: "Great employee". Stars: Stars.Five ReviewerId: "EMP001" ReviewedId: "EMP002"	O método AddReview retorna um OkObjectResult com a mensagem "Review added successfully" e a avaliação é adicionada à	Passed
TC002	Adicionar uma review com um revisor inválido	Base de dados em memória configurada e dados iniciais (empresa e funcionários) definidos	ReviewerId deve ser dado inválido.	O método AddReview retorna um NotFoundObjectResult com a mensagem "Reviewer not found" e a avaliação não é adicionada à base de dados.	Passed
TC003	Adicionar uma review cujo funcionário a ser avaliado é inválido	Base de dados em memória configurada e dados iniciais (empresa e funcionários) definidos	ReviewedId deve ser dado inválido	O método AddReview retorna um NotFoundObjectResult com a mensagem "Reviewed Employee not found"	Passed

OfferControllerTest

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC004	Adicionar uma oferta	A Base de dados configurada com uma empresa e um projeto	OfferId: "1", CompanyId: "1", ProjectId: "1", Date: DateTime.Now, Status: Status.PENDING, Description: "Description 1"	O método SendOffer retorna um OkObjectResult com a mensagem "Offer sent successfully"	Passed
TC005	Enviar uma oferta com uma empresa inválida	A Base de dados em memória configurada com uma empresa e um projeto	OfferId: "1", CompanyId: "invalid", ProjectId: "1", Date: DateTime.Now, Status: Status.PENDING, Description: "Description 1"	O método SendOffer retorna um NotFoundObjectResult com a mensagem "Company not found"	Passed
TC006	Enviar uma oferta com um projeto inválido	A Base de dados em memória configurada com uma empresa e um projeto	OfferId: "1", CompanyId: "1", ProjectId: "invalid", Date: DateTime.Now, Status: Status.PENDING, Description: "Description 1"	O método SendOffer retorna um NotFoundObjectResult com a mensagem "Project not found"	Passed

Project ControllerTest

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC007	Atribuir um funcionário a um projeto válido	Base de dados em memória configurada com dados iniciais	EmployeeId: "EMP001", ProjectId: "PROJ01"	O método AssignEmployee retorna um OkObjectResult com a mensagem "Employee assigned to project successfully."	Passed
TC008	Tentar atribuir um funcionário já atribuído ao mesmo projeto	Base de dados com funcionário já atribuído ao projeto	EmployeeId: "EMP001", ProjectId: "PROJ01"	O método AssignEmployee retorna um ConflictObjectResult com a mensagem "Employee is already assigned to this project."	Passed
TC009	Remover um funcionário que não está atribuído a um projeto	Base de dados com projeto e funcionário existentes	EmployeeId: "EMP002", ProjectId: "PROJ02"	O método RemoveEmployee retorna um NotFoundObjectResult com a mensagem "Employee not found."	Passed
TC010	Remover um funcionário de um projeto ao qual está atribuído	Base de dados com funcionário atribuído ao projeto	EmployeeId: "EMP001", ProjectId: "PROJ01"	O método RemoveEmployee retorna um OkObjectResult com a mensagem "Employee removed from project successfully."	Passed
TC011	Tentar atribuir um funcionário a um projeto com	Base de dados em memória configurada	EmployeeId: null, ProjectId: "PROJ01"	O método AssignEmployee retorna um BadRequestObjectResult	Passed

	EmployeeId nulo	com dados iniciais		com a mensagem "Employee ID cannot be null or empty."	
TC012	Tentar atribuir um funcionário a um projeto com ProjectId nulo	Base de dados em memória configurada com dados iniciais	EmployeeId: "EMP001", ProjectId: null	O método AssignEmployee retorna um BadRequestObjectResult com a mensagem "Project ID cannot be null or empty."	Passed
TC013	Tentar remover um funcionário de um projeto com EmployeeId nulo	Base de dados em memória configurada com dados iniciais	EmployeeId: null, ProjectId: "PROJ01"	O método RemoveEmployee retorna um BadRequestObjectResult com a mensagem "Employee ID cannot be null or empty."	Passed
TC014	Tentar remover um funcionário de um projeto com ProjectId nulo	Base de dados em memória configurada com dados iniciais	EmployeeId: "EMP001", ProjectId: null	O método RemoveEmployee retorna um BadRequestObjectResult com a mensagem "Project ID cannot be null or empty."	Passed
TC015	Aceitar uma oferta válida para um projeto	Base de dados em memória configurada com dados iniciais	CompanyId: "1", ProjectId: "PROJ01", OfferId: "OFFER01"	AcceptOffer retorna um OkObjectResult com a mensagem "Offer accepted successfully" e o status da oferta é atualizado para ACCEPTED.	Passed
TC016	Aceitar oferta com empresa inválida	Base de dados em memória configurada com dados iniciais	CompanyId: "invalid", ProjectId: "PROJ01", OfferId: "OFFER01"	AcceptOffer retorna um NotFoundObjectResult com a mensagem "Company not found"	Passed

TC017	Aceitar oferta com projeto inválido	Base de dados em memória configurada com dados iniciais	CompanyId: "1", ProjectId: "invalid", OfferId: "OFFER01"	AcceptOffer retorna um NotFoundObjectResult com a mensagem "Project not found".	Passed
TC018	Aceitar oferta com OfferId inválido	Base de dados em memória configurada com dados iniciais	CompanyId: "1", ProjectId: "PROJ01", OfferId: "invalid"	AcceptOffer retorna um NotFoundObjectResult com a mensagem "Offer not found".	Passed
TC019	Rejeitar uma oferta válida para um projeto	Base de dados em memória configurada com dados iniciais	CompanyId: "1", ProjectId: "PROJ01", OfferId: "OFFER01"	RejectOffer retorna um OkObjectResult com a mensagem "Offer rejected successfully" e o status da oferta é atualizado para DENIED.	Passed
TC020	Rejeitar oferta com empresa inválida	Base de dados em memória configurada com dados iniciais	CompanyId: "invalid", ProjectId: "PROJ01", OfferId: "OFFER01"	RejectOffer retorna um NotFoundObjectResult com a mensagem "Company not found".	Passed
TC021	Rejeitar oferta com projeto inválido	Base de dados em memória configurada com dados iniciais	CompanyId: "1", ProjectId: "invalid", OfferId: "OFFER01"	RejectOffer retorna um NotFoundObjectResult com a mensagem "Project not found".	Passed
TC022	Rejeitar oferta com OfferId inválido	Base de dados em memória configurada com dados iniciais	CompanyId: "1", ProjectId: "PROJ01", OfferId: "invalid"	RejectOffer retorna um NotFoundObjectResult com a mensagem "Offer not found".	Passed
TC023	Validar funcionamento	Base de dados em	Dados iniciais (povoados	Asserções para verificar se todos os getters	Passed

	dos getters e setters de Project	memória configurada, classe Project criada	aleatoriamente) ProjectId: Name: Description: Budget Status: StartDate: EndDate: CompanyId: Company: { }	retornam os valores esperados para as propriedades da instância project inicial. Todos os asserts finais validam se os novos valores foram aplicados corretamente.	
--	----------------------------------	--	---	--	--

CompanyControllerTest (Register)
AuthControllerTest(Login)

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC024	Registar uma nova empresa com sucesso	Base de dados em memória configurada	Name: "New Company", Address: "123 Street", Email: "newcompany@example.com", Contact: 123456789, TaxId: 123456789, Sector: "Sector", Plan: SubscriptionPlan.Premium, Password: "SecurePassword123"	O método CreateCompany retorna OkObjectResult com a mensagem "Company registered successfully" e um token não nulo é gerado.	Passed
TC025	Registar empresa com email duplicado	Base de dados em memória configurada com	Name: "New Company", Address: "456 Avenue", Email: "duplicate@example.com" (email já existente), Contact: 987654321, TaxId:	O método CreateCompany retorna BadRequestObjectResult com a mensagem "A company with	Passed

		uma empresa já registrada	987654321, Sector: "Sector", Plan: SubscriptionPlan.Premium, Password: "SecurePassword456"	this email already exists."	
TC026	Registrar empresa com plano de assinatura inválido	Base de dados em memória configurada	Name: "Invalid Subscription Plan Company", Address: "123 Street", Email: "invalidplan@example.com", Contact: 123456789, TaxId: 123456789, Sector: "Sector", Plan: (SubscriptionPlan)999 (plano inválido), Password: "SecurePassword123"	O método CreateCompany retorna BadRequestObjectResult com a mensagem "Invalid subscription plan."	Passed
TC027	Registrar empresa com formato de email inválido	Base de dados em memória configurada	Name: "Invalid Email Company", Address: "789 Boulevard", Email: "invalid-email" (email inválido), Contact: 123456789, TaxId: 123456789, Sector: "Sector", Plan: SubscriptionPlan.Premium, Password: "SecurePassword123"	O método CreateCompany retorna BadRequestObjectResult com a mensagem "Invalid email format."	Passed
TC028	Login_ValidCredentials_ReturnsToken retorna um token de autenticação para credenciais válidas	Base de dados em memória com utilizado	LoginDto com Email e Password corretos	Deve retornar um OkObjectResult contendo um token JWT não vazio.	Passed

		res válidos			
--	--	----------------	--	--	--

Company Controller Test

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC029	Adicionar novo projeto com sucesso	Base de dados em memória configurada com empresa ativa	CompanyId: "1", Name: "New Project1", Description: "New Project Description", Budget: 5000, StartDate: DateTime.Now, EndDate: DateTime.Now.AddMonths(1)	Método AddProject retorna OkObjectResult com mensagem "Project added successfully". Projeto adicionado corretamente ao base de dados com os detalhes fornecidos.	Passed
TC030	Retorna NotFound ao adicionar projeto para empresa inexistente	Base de dados em memória sem empresa com ID "999"	CompanyId: "999", Name: "New Project", Description: "New Project Description", Budget: 5000, StartDate: DateTime.Now, EndDate: DateTime.Now.AddMonths(1), EmployeeIds: ["EMP01"]	Método AddProject retorna NotFoundObjectResult com a mensagem "Company not found".	Passed
TC031	Retorna NotFound ao adicionar projeto com funcionário inexistente	Base de dados em memória sem funcionário com ID "NONEXISTENT_EMP" para a empresa com ID "1"	CompanyId: "1", Name: "New Project", Description: "New Project Description", Budget: 5000, StartDate: DateTime.Now, EndDate: DateTime.Now.AddMonths(1), EmployeeIds: ["NONEXISTENT_EMP"]	Método AddProject retorna NotFoundObjectResult com a mensagem "Employee with ID NONEXISTENT_EMP not found".	Passed

TC03 2	Editar projeto existente com sucesso	Base de dados em memória com projeto existente com ID "PROJ01" para a empresa com ID "1"	ProjectId: "PROJ01", Name: "Updated Project", Description: "Updated Project Description", Budget: 10000, StartDate: DateTime.Now, EndDate: DateTime.Now.AddMonths(2)	Método EditProject retorna OkObjectResult com a mensagem "Project updated successfully". Projeto é atualizado corretamente no base de dados com os novos valores.	Passado
TC03 3	Remove projeto existente com sucesso	Base de dados em memória com projeto com ID "PROJ01" associado a uma empresa existente	ProjectId: "PROJ01"	Método RemoveProject retorna OkObjectResult com a mensagem "Project removed successfully". O projeto com ProjectId: "PROJ01" não deve mais existir no base de dados (dbContext.Projects).	Passado
TC03 4	Editar projeto existente com sucesso	Base de dados em memória com projeto com ID "PROJ01"	ProjectDto: Nome="Updated Project", Descrição="Updated Project Description", Budget=10000	EditProject deve retornar OkObjectResult com a mensagem "Project updated successfully". Os detalhes do projeto com ProjectId: "PROJ01" devem ser atualizados no base de dados.	Passado
TC03 5	Visualizar projeto existente com sucesso	Base de dados em memória com projeto	ProjectId: "PROJ01"	ViewProject deve retornar OkObjectResult com os dados do projeto,	Passado

		com ID "PROJ01"		incluindo o nome "New Project".	
TC03 6	Adicionar funcionário a uma empresa existente	Base de dados em memória com empresa e projeto existente	EmployeeDto: Nome="Employee 1", Nif=123456789, Email="employee@example.com"	AddEmployee deve retornar OkObjectResult com a mensagem "Employee created successfully". O funcionário deve ser salvo no base de dados e associado à empresa especificada.	Passed
TC03 7	Remover funcionário existente com sucesso	Base de dados em memória com empresa e funcionário existente	EmployeeId: "E1"	RemoveEmployee deve retornar OkObjectResult com a mensagem "Employee removed successfully". O funcionário com ID "E1" não deve mais existir no base de dados.	Passed
TC03 8	Editar funcionário existente com sucesso	Base de dados em memória com empresa e funcionário existente	EmployeeDto: Nome="Updated Employee", Nif=987654321, Email="updated@example.com"	EditEmployee deve retornar OkObjectResult com a mensagem "Employee updated successfully". Os detalhes do funcionário com EmployeeId: "1" devem ser atualizados no base de dados.	Passed

TC039	GetPlanPrice e retorna o preço correto para planos conhecidos		SubscriptionPlan.Premium ou SubscriptionPlan.Basic	O método GetPlanPrice deve retornar o preço correto correspondente ao plano especificado.	Passado
TC040	GetPlanPrice e lança ArgumentOutOfRangeException para um plano desconhecido		SubscriptionPlan com valor inválido (ex: 999)	O método GetPlanPrice deve lançar uma exceção ArgumentOutOfRangeException ao ser passado um valor inválido de plano.	Passado
TC041	Propriedades de Payment funcionam corretamente e com getters e setters		Instância Payment com dados atribuídos	Todas as propriedades (PaymentID, Value, Date, CompanyID, Company, Plan) devem ser recuperadas e alteradas corretamente.	Passado
TC042	Propriedades de Company funcionam corretamente e com getters e setters		Instância Company com dados atribuídos	Todas as propriedades (CompanyID, Name, Address, Email, Contact, TaxID, Sector, Plan, Password) devem ser recuperadas e alteradas corretamente.	Passado
TC043	ListAllEmployeesTest retorna uma lista de	Base de dados em memória.	CompanyID válido com pagamentos	Deve retornar OkObjectResult contendo uma lista com o pagamento	Passado

	funcionários para uma empresa existente			adicionado, verificando o campo PaymentID.	
TC04 4	ListPaymentHistory_ShouldReturnOk_WhenPaymentsExist retorna histórico de pagamentos quando há pagamentos	Base de dados em memória .	CompanyID válido com pagamentos	Deve retornar OkObjectResult contendo uma lista com o pagamento adicionado, verificando o campo PaymentID.	Passed
TC04 5	UpgradePlan_ShouldReturnOk_WhenPlansUpgraded atualiza o plano com sucesso	Base de dados em memória	CompanyID válido e novo SubscriptionPlan	Deve retornar OkObjectResult e atualizar o SubscriptionPlan para o novo plano especificado.	Passed
TC04 6	UpgradePlan_ShouldReturnBadRequest_WhenPlansSame falha ao tentar atualizar para o mesmo plano	Base de dados em memória com plano Premium	CompanyID válido e plano atual	Deve retornar BadRequestObjectResult com mensagem "Subscription plan already upgraded".	Passed
TC04 7	UpgradePlan_ShouldReturnNotFound_WhenCompanyDoes	Base de dados em memória sem a	CompanyID inexistente	Deve retornar NotFoundObjectResult com a mensagem "Company not found".	Passed

	NotExist falha quando a empresa não existe	empresa especificada			
TC04 8	ListAllProjectsFromCompany_ShouldReturnOk_WhenProjectsExist retorna projetos para uma empresa existente	Base de dados em memória	CompanyID válido com projetos	Deve retornar OkObjectResult com a lista de projetos da empresa.	Passed
TC04 9	ListAllProjectsFromCompany_ShouldReturnNotFound_WhenCompanyDoesNotExist falha quando empresa não existe ListAllProjectsFromCompany_ShouldReturnNotFound_WhenCompanyDoesNotExist falha quando empresa não existe	Base de dados em memória sem a empresa especificada	CompanyID inexistente	Deve retornar NotFoundObjectResult com mensagem "Company not found".	Passed

TC050	ListAllProjectsFromCompany_ShouldReturnOk_WhenNoProjectsExist retorna sucesso quando não há projetos	Base de dados em memória com empresa sem projetos	CompanyID válido e sem projetos	Deve retornar OkObjectResult com mensagem "No projects found for this company".	Passe d
TC051	CancelSubscriptionTest cancela a assinatura de uma empresa existente	Base de dados em memória com empresa ativa e plano válido	CompanyID válido	Deve retornar OkObjectResult com mensagem "Subscription cancelled successfully" e definir o plano da empresa como SubscriptionPlan.None	Passe d
TC052	GetProjectById_ShouldReturnOk_WhenProjectExists retorna detalhes de projeto para uma empresa válida	Base de dados em memória	CompanyID e ProjectID válidos	Deve retornar OkObjectResult com detalhes do projeto, validando o nome do projeto.	Passe d
TC053	GetProjectById_ShouldReturnNotFound_WhenProjectDoesNotExist falha quando	Base de dados em memória com empresa válida	CompanyID válido e ProjectID inválido	Deve retornar NotFoundObjectResult com mensagem "Project not found or does not belong to the specified company".	Passe d

	projeto não existe				
TC054	GetProjectById_ShouldReturnNotFound_WhenCompanyDoesNotExist falha quando empresa não existe	Base de dados em memória	CompanyID inexistente	Deve retornar NotFoundObjectResult com mensagem "Company not found".	Passed

ChatController Test

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC055	Testar a criação de um novo chat	A base de dados, na memória deve conter um projeto com ProjectId igual a "2"	ChatId = "1", ProjectId = "2"	O método deve retornar um OkObjectResult com a mensagem "Chat created successfully"	Passed
TC056	Testar getters e setters das propriedades de Chat	Objeto Chat inicializado com ChatId, ProjectId, lista de Messages,	ChatId = "C12345", ProjectId = "P12345", lista de mensagens, projeto associados	Verificar que ChatId e ProjectId correspondem aos valores esperados, que a lista Messages tem 2 mensagens, e o Project possui Name = "Project 1".	Passed
TC057	Testar a exclusão de um chat existente	Base de dados em memória deve conter um chat com ChatId igual a "1"	ChatId = "1"	O método deve retornar um OkObjectResult com a mensagem	Passed

				"Chat deleted successfully"	
TC058	Testar a exclusão de um chat inexistente com ChatId inválido	A base de dados não contém um chat com o ChatId = .	ChatId = "non-existing-id"	O método DeleteChat retorna um NotFoundObjectResult com a mensagem "Chat not found".	Passed

MessageControllet Test

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC059	Validar getters e setters das propriedades de Message	Objeto Message inicializado com MessageId, Text, Date, EmployeeId, Employee, ChatId e Chat	MessageId = "M12345", Text = "Hello, World!", Date = new DateTime(2023, 10, 1), EmployeeId = "E12345"	Propriedades do objeto Message retornam os valores atribuídos corretamente.	Passed
TC060	Enviar uma nova mensagem válida	Objeto MessageDto criado com dados válidos;	MessageId = "1", Text = "Hello, World!",	O método SendMessage retorna CreatedAtAction	Passed

			EmployeeId = "user1", ChatId = "1"	nResult com ActionName para GetMessages, RouteValues com chatId = "1", e objeto Message com Text = "Hello, World!".	
TC061	Apagar uma mensagem com MessageId existente	Mensagem com MessageId = "2" existente na base de dados, associada a um Chat	MessageId = "2"	O método DeleteMessage e retorna OkObjectResult com a mensagem "Message deleted successfully".	Passed
TC062	Apagar uma mensagem com MessageId inexistente	Base de dados in-memory inicializada sem a mensagem com MessageId = "non-existing-id"	MessageId = "non-existing-id"	O método DeleteMessage e retorna NotFoundObjectResult, indicando que a mensagem com MessageId = "non-existing-id" não foi encontrada na base de dados.	Passed
TC063	Listar todas as mensagens de um chat específico	Chat existente com ChatId = "2" associado a duas mensagens na base de dados in-memory	ChatId = "2"	O método GetMessages retorna OkObjectResult contendo uma lista de duas mensagens.	Passed

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC064	Validar getters e setters das propriedades do Employee	Instância de Employee criada com valores definidos para todas as propriedades.	EmployeeID="E12345", Name="John Doe", Profession="Developer", NIF=123456789, Email="john.doe@example.com",	Retorna os valores iniciais e alterados para todas as propriedades de Employee, conforme esperado.	Passed
TC065	Listar todos os funcionários usando ListAllEmployees	Base de dados em memória com dois funcionários adicionados através do método SeedData	Nenhum	Retorno de uma lista com 2 funcionários, contendo John Doe (ID="1") e Jane Doe (ID="2")	Passed
TC066	Verificar se GetEmployeeById retorna um funcionário existente	Base de dados em memória com o funcionário John Doe (ID="1") adicionado via método SeedData	EmployeeID="1"	Retorno do funcionário com EmployeeID="1" e Name="John Doe"	Passed
TC067	Verificar se GetEmployeeById retorna NotFound para um funcionário inexistente	Base de dados em memória sem funcionários adicionados via método SeedData.	EmployeeID="999"	Retorna NotFoundObjectResult com a mensagem "Employee not found"	Passed
TC068	Verificar se GetEmployeeProjects retorna corretamente os projetos associados a um funcionário	Base de dados em memória com o funcionário John Doe (EmployeeID="1") e dois projetos associados via método SeedData.	EmployeeID="1"	Retorna OkObjectResult, uma lista de 2 projetos: Project 1 (ProjectId="1") e Project 2 (ProjectId="2")	Passed
TC069	Verificar se GetEmployeeProjects retorna	Base de dados em memória com o funcionário Jane Doe (EmployeeID="2") sem	EmployeeID="2"	Retorna NotFoundObjectResult com a	Passed

	NotFound para um funcionário inexistente	projetos associados, adicionado via SeedData		mensagem "No projects found for this employee."	
TC070	Verificar se GetEmployeeProjects retorna NotFound para um funcionário sem projetos associados	Base de dados em memória com o funcionário Jane Doe (ID="2") sem projetos associados, adicionado via método SeedData.	EmployeeID="2"	Retorno NotFoundObjectResult com a mensagem "No projects found for this employee."	Passed

TaskControllerTest

Test Case ID	Test Case Scenario	Pre-Conditions	Test Data	Expected Results	Status
TC071	Verificar se os getters e setters das propriedades da classe Task funcionam corretamente	Instância de Task criada com valores definidos para todas as propriedades.	TaskId="T12345", TitleName="Task 1", Description="Description of Task 1", Status=INPROGRESS, Date="2023-10-01", ProjectId="P12345", EmployeeId="E12345", FilePath="/path/to/file"	O objeto Task deve ser inicializado corretamente com os valores especificados e, depois, atualizado para os novos valores de propriedades	Passed
TC072	Verificar se a criação de uma nova tarefa retorna	Base de dados em memória com projetos e funcionários iniciais através SeedData	TitleName="New Task", Description="New	Retorna OkObjectResult com a	Passed

	uma resposta de sucesso com a tarefa criada		Task Description", Date=DateTime.Now, FilePath="path/to/file", ProjectId="1", EmployeeId="1", Status=PENDING	mensagem "Task created successfully"	
TC073	Verificar se a consulta de todas as tarefas retorna corretamente uma lista de todas as tarefas	Base de dados em memória com tarefas registradas via SeedData	Nenhum	Retorna OkObjectResult contendo uma lista com todas as tarefas cadastradas	Passed
TC074	Verificar se a consulta de uma tarefa por ID retorna corretamente a tarefa com os detalhes do título e descrição	Base de dados em memória com a tarefa com ID "c" adicionado via SeedData	TaskId="c"	Retorna OkObjectResult com a tarefa que possui ID "c", incluindo o título e a descrição da tarefa	Passed
TC075	Verificar se a atualização de uma tarefa retorna uma resposta de sucesso com a tarefa atualizada	Base de dados em memória com a tarefa com ID "d" já criada via SeedData	TaskId="d", TitleName="Updated Task", Description="Updated Task Description", Date=DateTime.Now, Status=PENDING, ProjectId="1", EmployeeId="1"	Retorna OkObjectResult com a mensagem "Task updated successfully".	Passed
TC076	Verificar se a exclusão de uma tarefa retorna uma resposta de sucesso com a tarefa deletada	Base de dados em memória com a tarefa com ID "e" já criada via SeedData	TaskId="e"	Retorna OkObjectResult com a mensagem "Task deleted successfully",	Passed
TC077	Verificar se a exclusão de uma tarefa não	Base de dados em memória sem a tarefa com ID "f"	TaskId="f"	Retorna NotFoundResult com a	Passed

	encontrada retorna o (NotFound)			mensagem "Task not found"	
TC078	Verificar se a atualização de uma tarefa não encontrada retorna o resultado (NotFound)	Base de dados em memória sem a tarefa com ID "i"	TaskId="i", TitleName="Updated Task", Description="Updated Task Description", Date=DateTime.Now, Status=PENDING, ProjectId="1", EmployeeId="1"	NotFoundObjectResult com a mensagem "Task not found"	Passed
TC079	Verificar se a consulta de uma tarefa por ID retorna NotFound quando a tarefa não existe	Base de dados em memória sem a tarefa com ID "j"	TaskId="j"	Retorna NotFoundResult com a mensagem "Task not found"	Passed
TC080	Verificar se a consulta de todas as tarefas retorna uma lista vazia quando não há tarefas cadastradas	Base de dados em memória sem tarefas registadas	Nenhum	Retorna OkObjectResult com uma lista vazia de tarefas	Passed
TC081	Verificar se a criação de uma tarefa retorna BadRequest quando os dados da tarefa são nulos	Base de dados sem dados	TaskDto=null	Retorna BadRequestObjectResult com a mensagem "Task is null"	Passed
TC082	Verificar se a atualização de uma tarefa retorna BadRequest quando o estado do modelo é inválido	Base de dados sem dados	TaskDto com o campo TitleName inválido (ex.: vazio ou não preenchido)	Retorna BadRequestObjectResult com os erros de validação, e o erro de "TitleName required".	Passed

11. Evidências de requests

11.1 Postman

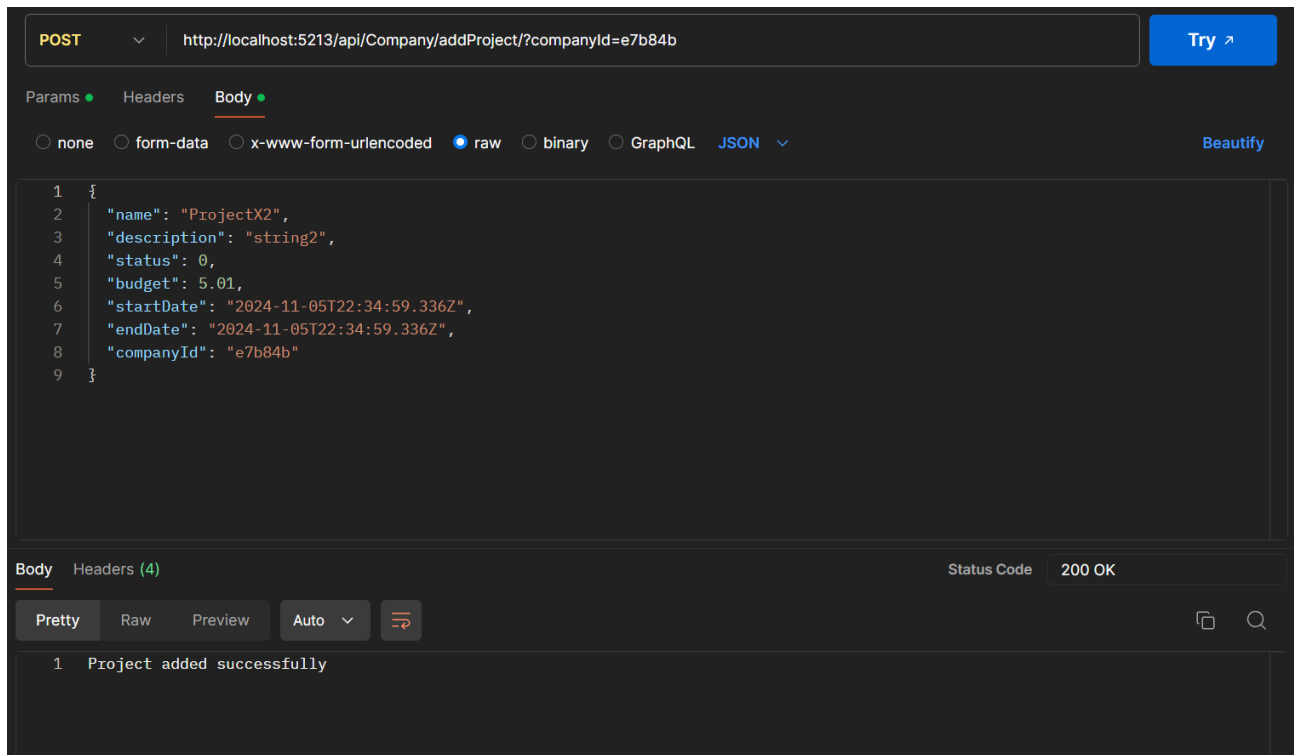


Figura 38 – Evidência de uso do PostMan

11.2 Swagger

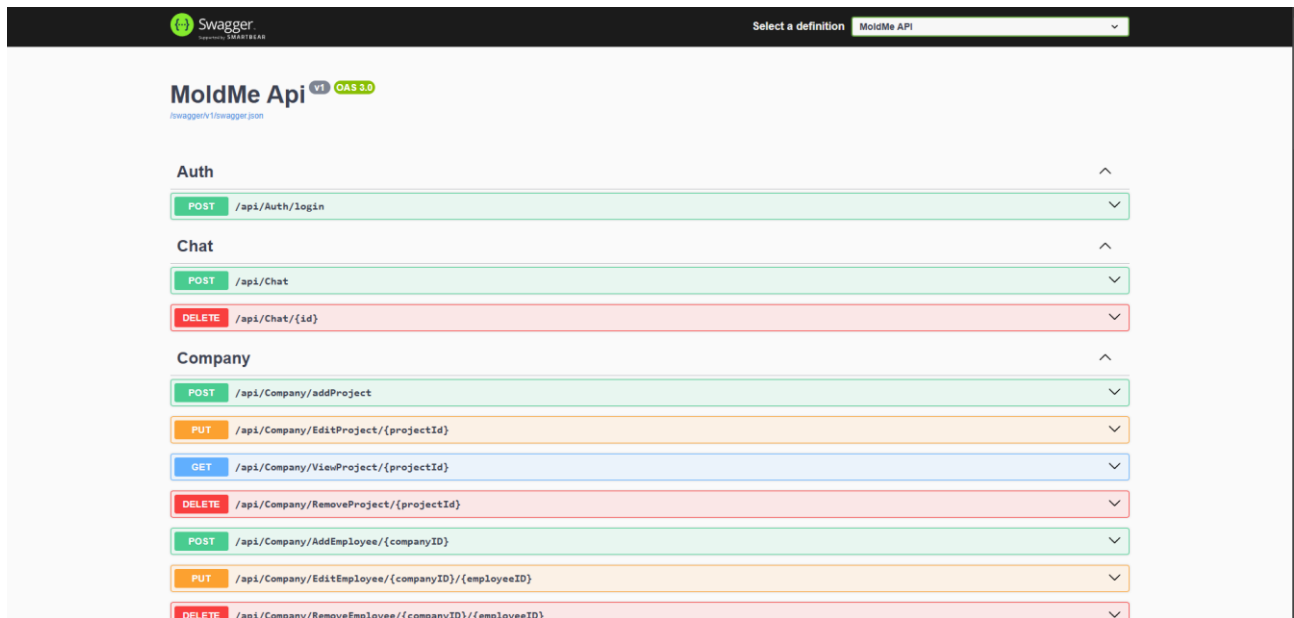


Figura 39 – Evidência de uso do Swagger

11.3 Scrum